EX.NO.1	CNN MODEL USING TENSOR FLOW
DATE: 09.07.26	

AIM:

To implement a basic Convolutional Neural Network (CNN) using TensorFlow/Keras for recognizing handwritten digits from the MNIST dataset and evaluate its classification accuracy.

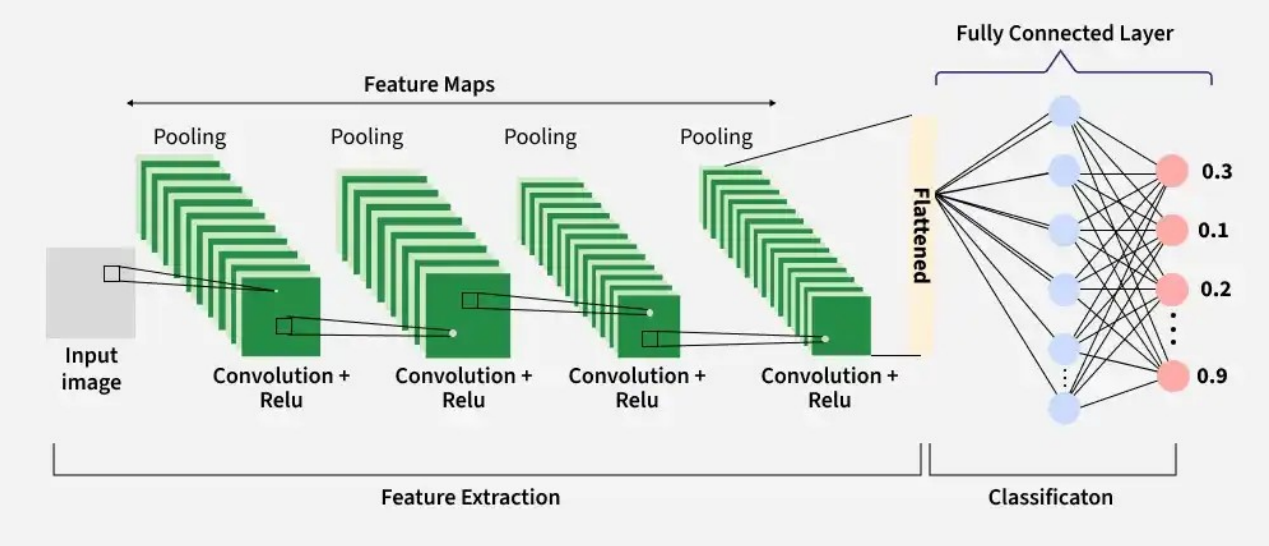
DESCRIPTION:

Convolutional Neural Network (CNN)

A Convolutional Neural Network (CNN) is a type of deep learning algorithm specifically designed for image processing and computer vision applications. Unlike tr
aditional neural networks, CNNs automatically learn important image features such as edges, textures, patterns, and shapes using convolution operations. These extracted features are then used to classify or recognize images with high accuracy. CNNs are widely used in applications such as handwritten digit recognition, face recognition, object detection, medical image analysis, autonomous vehicles, and biometric authentication.

CNN ARCHITECTURE:

The CNN architecture used in this experiment consists of the following layers:



WORKFLOW OF CNN ARCHITECTURE:

Input Image → Convolution + ReLU → Max Pooling → Convolution + ReLU → Max Pooling →

Flatten → Fully Connected (Dense) Layer → Softmax Output Layer → Predicted Digit (0–9)

IMPORT LIBRARIES:

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
```

```
import matplotlib.pyplot as plt
import numpy as np
```

PROGRAM:

Mount Google Drive:

```
from google.colab import drive
print("STEP 1 : MOUNTING GOOGLE DRIVE")
drive.mount('/content/drive')
print("Google Drive Mounted Successfully.")
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

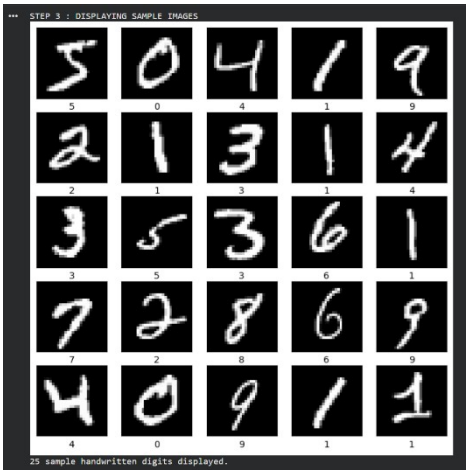
Load Dataset:

```
print("STEP 2 : LOADING MNIST DATASET")
(train_images, train_labels), (test_images, test_labels) = datasets.mnist.load_data()
print("Training Images :", train_images.shape)
print("Training Labels :", train_labels.shape)
print("Testing Images  :", test_images.shape)
print("Testing Labels  :", test_labels.shape)
print("\nNormalizing pixel values...")
train_images = train_images / 255.0
test_images = test_images / 255.0
print("Normalization Completed")
print("\nReshaping images to (28,28,1)...")
train_images = train_images.reshape((60000,28,28,1))
test_images = test_images.reshape((10000,28,28,1))
print("Training Shape :", train_images.shape)
print("Testing Shape  :", test_images.shape)
```

```
Dataset Loaded Successfully
Training Images : (60000, 28, 28)
Training Labels : (60000,)
Testing Images  : (10000, 28, 28)
Testing Labels  : (10000,)
```

Verify the Dataset:

```
print("STEP 3 : DISPLAYING SAMPLE IMAGES")
plt.figure(figsize=(8,8))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i].reshape(28,28), cmap='gray')
    plt.xlabel(train_labels[i])
plt.show()
print("25 sample handwritten digits displayed.")
```



Create the Convolutional Base:

```
print("STEP 4 : BUILDING CNN MODEL")
model = models.Sequential()
print("Adding Input Layer")
model.add(layers.Input(shape=(28,28,1)))
print("Adding First Convolution Layer")
model.add(layers.Conv2D(32,(3,3),activation='relu'))
print("Adding First MaxPooling Layer")
model.add(layers.MaxPooling2D((2,2)))
print("Adding Second Convolution Layer")
model.add(layers.Conv2D(64,(3,3),activation='relu'))
print("Adding Second MaxPooling Layer")
model.add(layers.MaxPooling2D((2,2)))
print("\nConvolutional Base Created Successfully.")
```

```
============================================================
STEP 4 : BUILDING CNN MODEL
============================================================
Adding Input Layer
Adding First Convolution Layer
Adding First MaxPooling Layer
Adding Second Convolution Layer
Adding Second MaxPooling Layer

Convolutional Base Created Successfully.
```

Display the Architecture:

```
print("STEP 5 : CNN ARCHITECTURE")
model.summary()
```

```
STEP 5 : CNN ARCHITECTURE
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0

```
Total params: 18,816 (73.50 KB)
Trainable params: 18,816 (73.50 KB)
Non-trainable params: 0 (0.00 B)
```

Add Dense Layers:

```
print("STEP 6 : ADDING DENSE LAYERS")
model.add(layers.Flatten())
print("Flatten Layer Added")
model.add(layers.Dense(64,activation='relu'))
print("Dense Layer (64 Neurons) Added")
model.add(layers.Dense(10,activation='softmax'))
print("Output Layer (10 Neurons) Added")
print("\nComplete CNN Model Created.")
```

```
STEP 6 : ADDING DENSE LAYERS
Flatten Layer Added
Dense Layer (64 Neurons) Added
Output Layer (10 Neurons) Added

Complete CNN Model Created.
```

Display Complete Model:

```
print("STEP 7 : COMPLETE MODEL SUMMARY")
model.summary()
```

```
================================================================
STEP 7 : COMPLETE MODEL SUMMARY
================================================================
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 64)	102,464
dense_1 (Dense)	(None, 10)	650

```
Total params: 121,930 (476.29 KB)
Trainable params: 121,930 (476.29 KB)
Non-trainable params: 0 (0.00 B)
```

Compile and Train the Model:

```
print("STEP 8 : COMPILING MODEL")
model.compile(
```

```
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
print("Optimizer : Adam")
print("Loss      : Sparse Categorical Crossentropy")
print("Metric    : Accuracy")
print("\nTraining Started...\n")
history = model.fit(
    train_images,
    train_labels,
    epochs=5,
    validation_data=(test_images,test_labels)
)
print("\nTraining Completed Successfully.")
```

```
STEP 8 : COMPILING MODEL
Optimizer : Adam
Loss      : Sparse Categorical Crossentropy
Metric    : Accuracy

Training Started...

Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 52s 27ms/step - accuracy: 0.9445 - loss: 0.2691 - val_accuracy: 0.9797 - val_loss: 0.0742
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 52s 28ms/step - accuracy: 0.9810 - loss: 0.0641 - val_accuracy: 0.9802 - val_loss: 0.0622
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 49s 26ms/step - accuracy: 0.9847 - loss: 0.0500 - val_accuracy: 0.9862 - val_loss: 0.0460
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 51s 27ms/step - accuracy: 0.9885 - loss: 0.0381 - val_accuracy: 0.9856 - val_loss: 0.0499
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 50s 27ms/step - accuracy: 0.9896 - loss: 0.0341 - val_accuracy: 0.9846 - val_loss: 0.0562

Training Completed Successfully.
```

Evaluate the Model:

```
print("STEP 9 : MODEL EVALUATION")
test_loss, test_acc = model.evaluate(test_images,test_labels)
print("\nTest Loss :", test_loss)
print("Test Accuracy :", round(test_acc*100,2),"%")
```

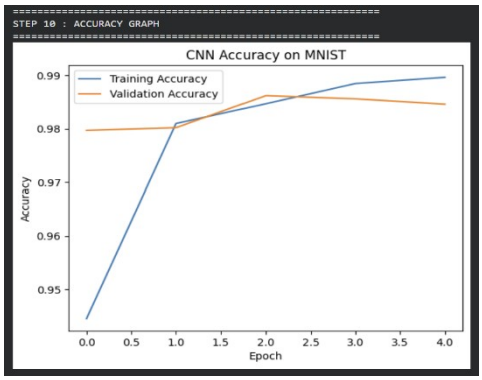
```
============================================================
STEP 9 : MODEL EVALUATION
============================================================
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9846 - loss: 0.0562

Test Loss : 0.056191399693489075
Test Accuracy : 98.46 %
```

Plot Accuracy Graph:

```
print("STEP 10 : ACCURACY GRAPH")
plt.plot(history.history['accuracy'],label='Training Accuracy')
plt.plot(history.history['val_accuracy'],label='Validation Accuracy')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CNN Accuracy on MNIST")
plt.legend()
```

```
plt.show()
```



Predict a Test Image:

```
print("STEP 11 : PREDICTION")
predictions = model.predict(test_images)
index = 0
predicted_digit = np.argmax(predictions[index])
print("Predicted Digit :", predicted_digit)
print("Actual Digit    :", test_labels[index])
print("\nProbability for each digit\n")
for i,p in enumerate(predictions[index]):
print(f"Digit {i} : {p:.4f}")
```

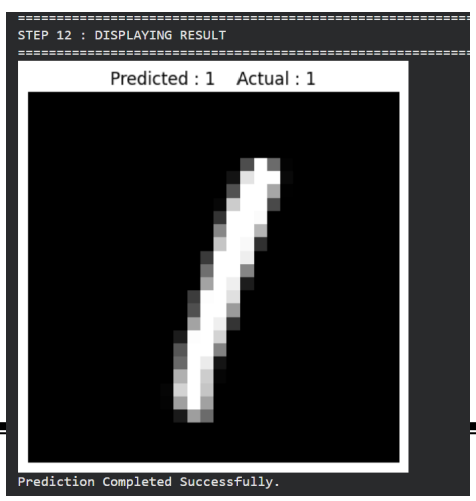
```
============================================================
STEP 11 : PREDICTION
============================================================
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step
Predicted Digit : 1
Actual Digit    : 1

Probability for each digit

Digit 0 : 0.0000
Digit 1 : 1.0000
Digit 2 : 0.0000
Digit 3 : 0.0000
Digit 4 : 0.0000
Digit 5 : 0.0000
Digit 6 : 0.0000
Digit 7 : 0.0000
Digit 8 : 0.0000
Digit 9 : 0.0000
```

Display Predicted Image:

```
print(“STEP 12 : DISPLAYING RESULT”)
plt.imshow(test_images[index].reshape(28,28),cmap=’gray’)
plt.title(
   f”Predicted : {predicted_digit}    Actual : {test_labels[index]}”
)
plt.axis(“off”)
plt.show()
print(“Prediction Completed Successfully.”)
```



EX.NO.2a	CNN VERIFICATION
DATE: 16.07.26	

RESULT:

The Convolutional Neural Network (CNN) was successfully implemented using the MNIST handwritten digit dataset.

AIM:

To verify the performance of the trained Convolutional Neural Network (CNN) model by predicting handwritten digit images from the MNIST test dataset and comparing the predicted labels with the actual labels.

ALGORITHM:

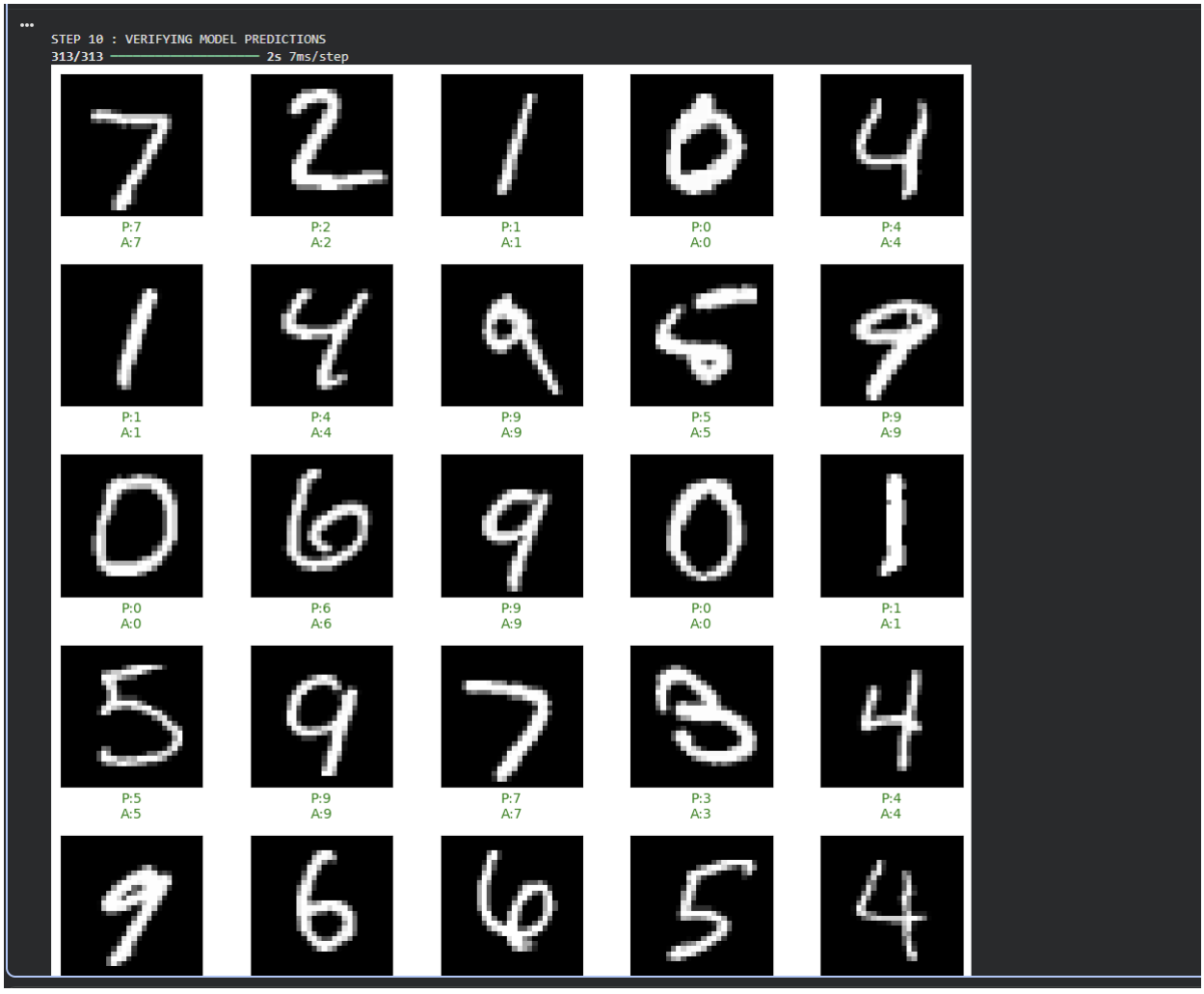
1. Display the title indicating the verification step.
2. Create a figure of size 10 × 10 to display multiple images.
3. Use the trained CNN model to predict the class probabilities of the test images.
4. Loop through the first 25 test images.
5. Divide the figure into a 5 × 5 grid using subplots.
6. Remove the x-axis ticks, y-axis ticks, and grid for better visualization.
7. Display each test image in grayscale.
8. Determine the predicted class using argmax() on the prediction probabilities.
9. Retrieve the actual label from the test dataset.
10. Compare the predicted label with the actual label.
11. Display the predicted label (P) and actual label (A) below each image.
12. Show the label in green if the prediction is correct; otherwise, show it in red.
13. Display all 25 images and print a confirmation message indicating that prediction verification is complete.

PROGRAM:

```
print("\nSTEP 10 : VERIFYING MODEL PREDICTIONS")
plt.figure(figsize=(10,10))
predictions = model.predict(test_images)
for i in range(25):
   plt.subplot(5,5,i+1)
   plt.xticks([])
   plt.yticks([])
   plt.grid(False)
   plt.imshow(test_images[i].reshape(28,28), cmap='gray')
   predicted = np.argmax(predictions[i])
```

```
        actual = test_labels[i]
        color = "green" if predicted == actual else "red"
        plt.xlabel(f"P:{predicted}\nA:{actual}", color=color)
    plt.tight_layout()
    plt.show()
    print("Prediction Verification Completed.")
```

OUTPUT:



EX.NO.2b	CNN VALIDATION
DATE: 16.07.26	

RESULT:

The trained CNN model successfully predicted the handwritten digits from the MNIST test dataset.

AIM:

To validate the performance of the trained Convolutional Neural Network (CNN) model using the MNIST test dataset by calculating the test accuracy, generating a classification report, and displaying the confusion matrix.

ALGORITHM:

1. Import the required libraries: NumPy, Matplotlib, and evaluation metrics from Scikit-learn.
2. Display the title indicating the model validation step.
3. Use the trained CNN model to predict the class probabilities for the test images.
4. Convert the predicted probabilities into class labels using the argmax() function.
5. Compare the predicted labels with the actual test labels to calculate the overall test accuracy.
6. Display the test accuracy as a percentage.
7. Generate and display the Classification Report, which includes:
   - Precision
   - Recall
   - F1-score
   - Support for each digit class (0–9)
8. Generate the Confusion Matrix by comparing the actual and predicted labels.
9. Display the confusion matrix in both numerical and graphical forms.
10. Print a confirmation message indicating that model validation has been completed successfully.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
print("\nSTEP 8 : MODEL VALIDATION")
y_pred_prob = model.predict(test_images)
y_pred = np.argmax(y_pred_prob, axis=1)
accuracy = np.mean(y_pred == test_labels)
print(f"\nTest Accuracy : {accuracy*100:.2f}%")
print("\nClassification Report:")
print(classification_report(test_labels, y_pred))
```

```
cm = confusion_matrix(test_labels, y_pred)
print("\nConfusion Matrix:")
print(cm)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()
print("\nModel Validation Completed Successfully.")
```

OUTPUT:

```
STEP 8 : MODEL VALIDATION
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step

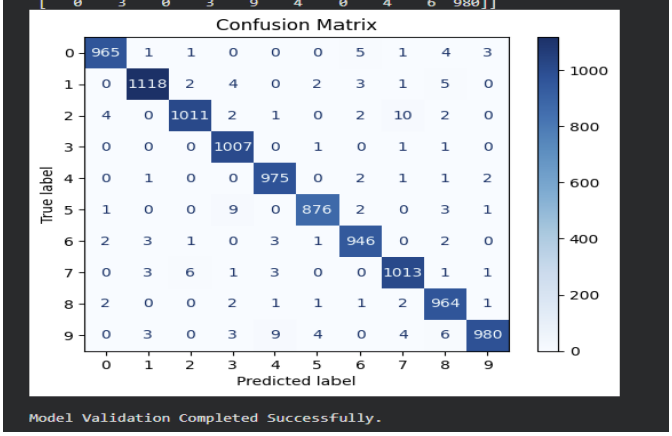
Test Accuracy : 98.55%

Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.98      0.99       980
           1       0.99      0.99      0.99      1135
           2       0.99      0.98      0.98      1032
           3       0.98      1.00      0.99      1010
           4       0.98      0.99      0.99       982
           5       0.99      0.98      0.99       892
           6       0.98      0.99      0.99       958
           7       0.98      0.99      0.98      1028
           8       0.97      0.99      0.98       974
           9       0.99      0.97      0.98      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```

```
Confusion Matrix:
[[ 965    1    1    0    0    0    5    1    4    3]
 [   0 1118    2    4    0    2    3    1    5    0]
 [   4    0 1011    2    1    0    2   10    2    0]
 [   0    0    0 1007    0    1    0    1    1    0]
 [   0    1    0    0  975    0    2    1    1    2]
 [   1    0    0    9    0  876    2    0    3    1]
 [   2    3    1    0    3    1  946    0    2    0]
 [   0    3    6    1    3    0    0 1013    1    1]
 [   2    0    0    2    1    1    1    2  964    1]
 [   0    3    0    3    9    4    0    4    6  980]]
```




```
Model Validation Completed Successfully.
```

EX.NO.3	CNN DETECTION
DATE: 03.08.26	

RESULT:

The trained CNN model was successfully validated using the MNIST test dataset.

AIM:

To develop and train a Convolutional Neural Network (CNN) for detecting and classifying synthetic images containing Heart, Star, and Diamond shapes, and to predict their bounding boxes.

ALGORITHM:

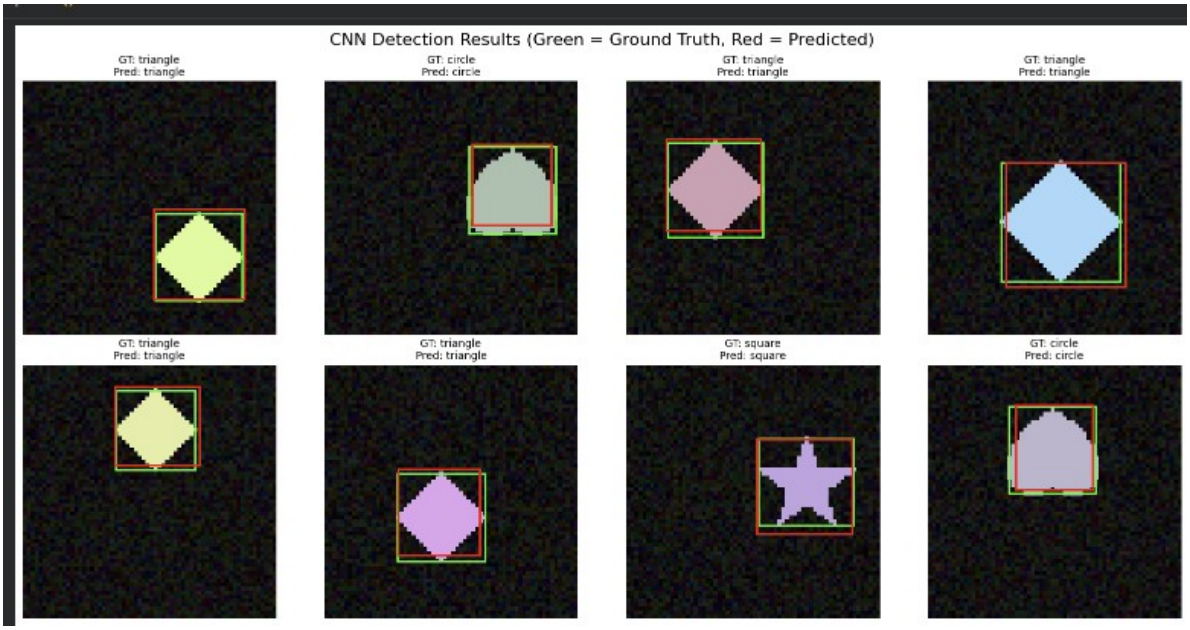
1. Import the required Python libraries.
2. Define the image size and shape classes (Heart, Star, Diamond).
3. Generate a synthetic dataset by randomly drawing one shape in each image.
4. Store the corresponding class label and normalized bounding box coordinates.
5. Build a CNN with shared convolutional layers.
6. Use one output layer for shape classification and another for bounding-box prediction.
7. Train the model using the generated dataset.
8. Evaluate the model using classification accuracy and bounding-box IoU.
9. Display the predicted class and bounding boxes on test images.

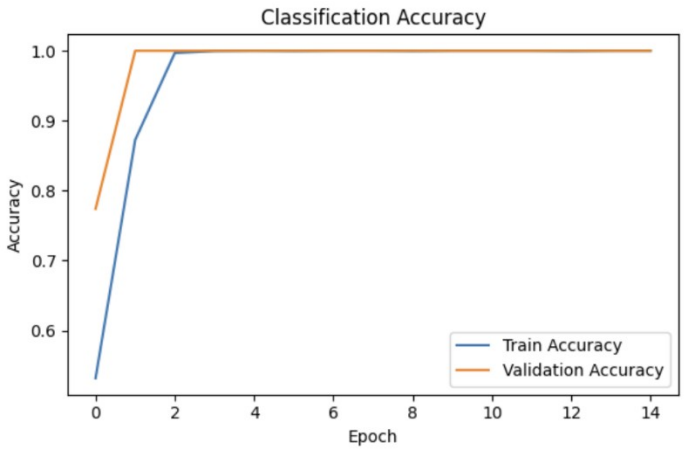
CODE:

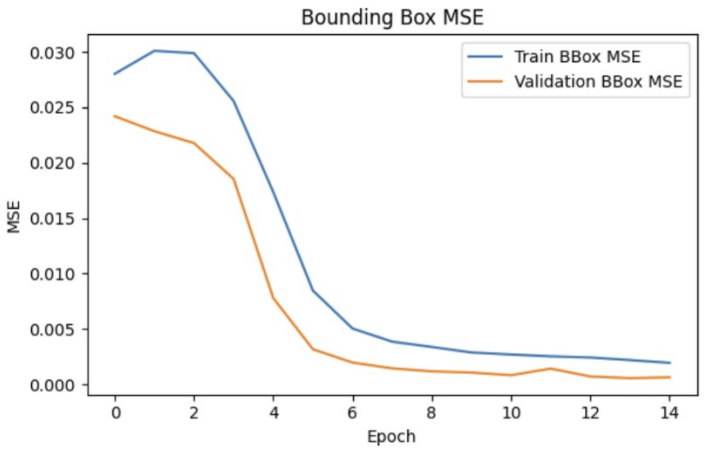
```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
IMG_SIZE = 64
CLASSES = ["Heart", "Star", "Diamond"]
NUM_CLASSES = len(CLASSES)
inputs = layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = layers.Conv2D(16, (3,3), activation='relu', padding='same')(inputs)
x = layers.MaxPooling2D((2,2))(x)
x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(x)
x = layers.MaxPooling2D((2,2))(x)
x = layers.Conv2D(64, (3,3), activation='relu', padding='same')(x)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.3)(x)
class_output = layers.Dense(NUM_CLASSES,
                            activation='softmax',
                            name='class_output')(x)
```

```
bbox_output = layers.Dense(4,
                           activation='sigmoid',
                           name='bbox_output')(x)
model = models.Model(inputs=inputs,
                     outputs=[class_output, bbox_output])
model.compile(
   optimizer='adam',
   loss={
      'class_output':'sparse_categorical_crossentropy',
      'bbox_output':'mse'
   },
   metrics={
      'class_output':'accuracy',
      'bbox_output':'mse'
   }
)
print("CNN Object Detection Model Created Successfully")
model.summary()
```

OUTPUT:







EX.NO.4	CNN SEGMENTATION
DATE: 10.09.26	

RESULT:

The CNN model successfully detected and classified the synthetic Heart, Star, and Diamond shapes.

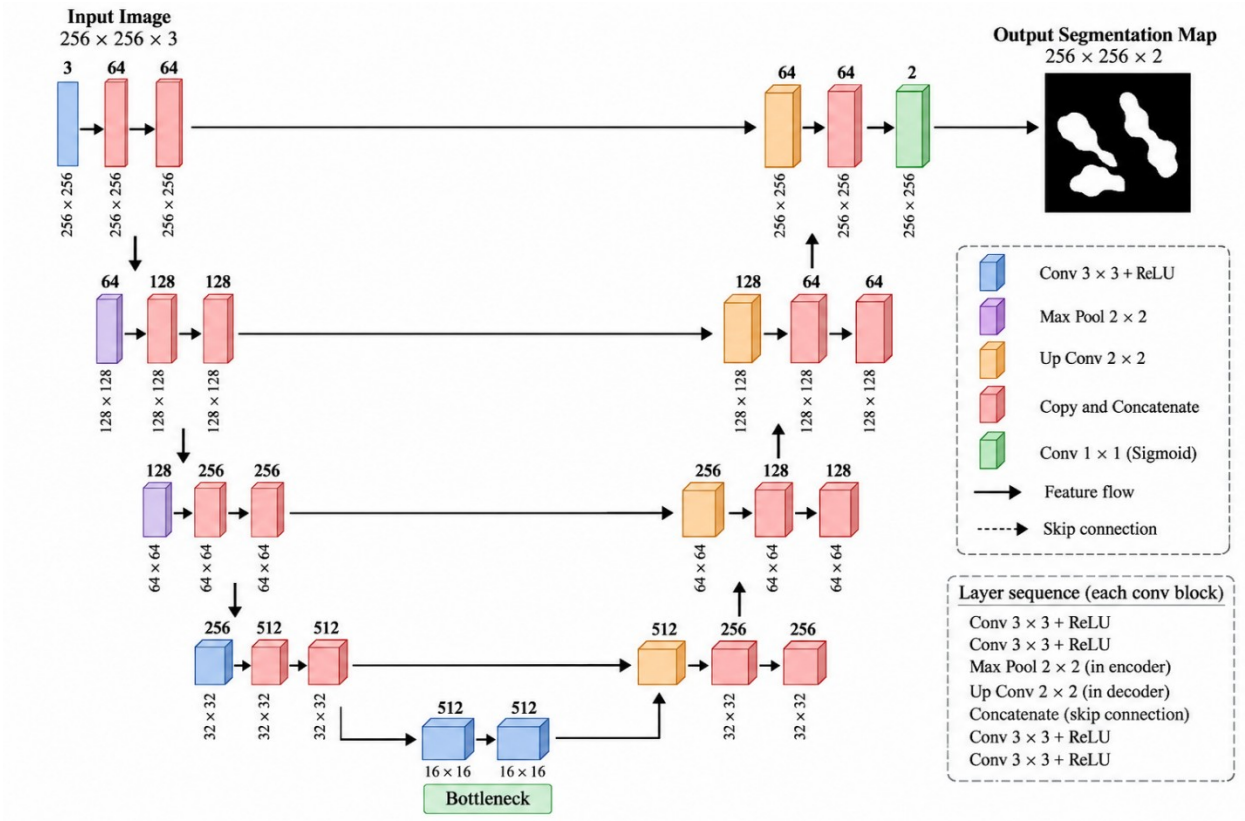
AIM:

To implement an image segmentation model using a Convolutional Neural Network (CNN) and generate a pixel-level segmentation mask for the objects present in an image.

ALGORITHM:

1. Start the program and import the required libraries such as NumPy, Matplotlib, and TensorFlow/Keras.
2. Define the image parameters, such as image size and number of samples.
3. Generate the dataset containing images and their corresponding segmentation masks.
4. Preprocess the images and masks into a suitable format for CNN input.
5. Split the dataset into training and testing sets.
6. Build the CNN model using convolution, pooling, upsampling, and output layers.
7. Compile the model using the Adam optimizer, binary cross-entropy loss, and accuracy metric.
8. Train the CNN model using the training images and their corresponding masks.
9. Evaluate the model using the test dataset.
10. Predict segmentation masks for the test images.
11. Display the original image, actual mask, and predicted mask for comparison.
12. Stop the program.

U – NET ARCHITECTURE :

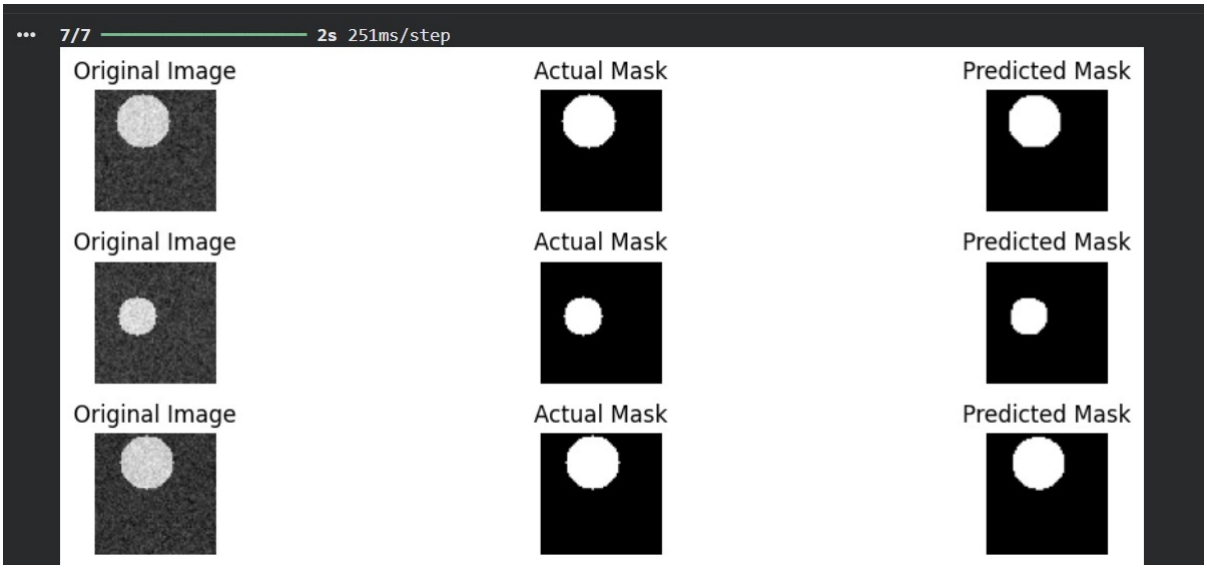


PROGRAM:

```
import os
import numpy as np
import matplotlib.pyplot as plt
import unet
from unet import UNet
image_size = 256
image_channels = 3
image_dir = "dataset"
epochs = 20
batch_size = 8
validation_data_size = 50
test_data_size = 200
train_data = 600
def visualize(image, mask, prediction_img=None):
    fig = plt.figure()
    fig.suptitle("Visualization", fontsize=15)
    ax = fig.add_subplot(1, 3, 1)
    ax.imshow(image)
    ax.set_title("Image")
    ax.axis("off")
    ax = fig.add_subplot(1, 3, 2)
    ax.imshow(
        np.squeeze(mask),
        cmap="gray"
    )
    ax.set_title("Original Mask")
    ax.axis("off")
```

```
        if prediction_img is not None:
            ax = fig.add_subplot(1, 3, 3)
            ax.imshow(
                np.squeeze(prediction_img),
                cmap="gray"
            )
            ax.set_title("Predicted Mask")
            ax.axis("off")
        plt.show()
    ax.set_xticks([])
    ax.set_yticks([])
    temp_data = NucleiDataGenerator(
        img_ids=train_data,
        img_size=image_size,
        batch_size=batch_size
    )
    image, mask = temp_data.__getitem__(0)
    print("Batch shape:", image.shape, mask.shape)
    visualize(
        image=image[2],
        mask=mask[2]
    )
```

OUTPUT:



RESULT:

The CNN-based image segmentation model was successfully implemented.

EX.NO.5	CNN Feature Extraction from Video Frames
DATE: 17.09.26	

AIM:

To extract spatial features from the frames of the UCF101 video dataset using a pre-trained Convolutional Neural Network (InceptionV3), so that every video frame is represented by a fixed-length 2048-dimensional feature vector that can be fed to a sequence model.

ALGORITHM:

1. Install TensorFlow Docs and download the sub-sampled UCF101 dataset (5 action classes).
2. Import the required libraries and define the hyper-parameters (IMG_SIZE = 224, MAX_SEQ_LENGTH = 20, NUM_FEATURES = 2048).
3. Read train.csv and test.csv that list the video names and their class labels.
4. Capture the frames of each video with OpenCV's VideoCapture(), crop the centre square, resize it to 224 × 224 and convert BGR to RGB.
5. Build the feature extractor using InceptionV3 pre-trained on ImageNet (include_top = False, pooling = "avg").
6. Convert the string class labels to integers using the StringLookup layer.
7. For every video, pass at most 20 frames through the CNN to obtain a (20, 2048) feature matrix; pad shorter videos with zeros and store a frame mask.
8. Display the shape of the extracted features and visualise the feature vectors.

ARCHITECTURE DIAGRAM:

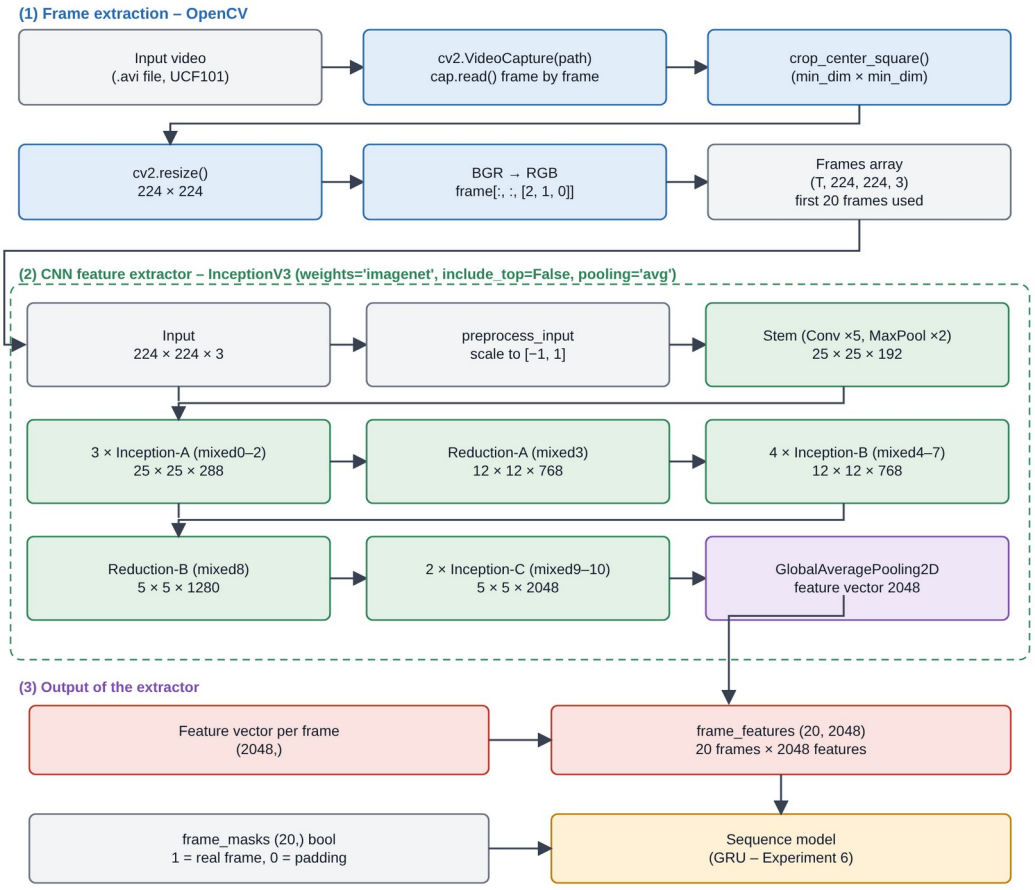


Fig. 5.1 Frame extraction and InceptionV3 feature extractor (CNN) for video frames

PROGRAM:

Install TensorFlow Docs and download the dataset:

```
!pip install -q git+https://github.com/tensorflow/docs
!wget -q https://github.com/sayakpaul/Action-Recognition-in-TensorFlow/releases/download/v1.0.0/
ucf101_top5.tar.gz
!tar xf ucf101_top5.tar.gz
```

Setup and hyper-parameters:

```
import os
import keras
from imutils import paths

import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import imageio
import cv2
from IPython.display import Image

IMG_SIZE = 224
BATCH_SIZE = 64
EPOCHS = 10

MAX_SEQ_LENGTH = 20
NUM_FEATURES = 2048
```

Data preparation:

```
train_df = pd.read_csv("train.csv")
test_df = pd.read_csv("test.csv")

print(f"Total videos for training: {len(train_df)}")
print(f"Total videos for testing: {len(test_df)}")

train_df.sample(10)
```

```
Total videos for training: 594
Total videos for testing: 224
```

	video_name	tag
462	v_ShavingBeard_g23_c06.avi	ShavingBeard
487	v_TennisSwing_g09_c05.avi	TennisSwing
194	v_PlayingCello_g19_c04.avi	PlayingCello
476	v_ShavingBeard_g25_c06.avi	ShavingBeard
524	v_TennisSwing_g14_c07.avi	TennisSwing
189	v_PlayingCello_g18_c06.avi	PlayingCello
573	v_TennisSwing_g22_c04.avi	TennisSwing
463	v_ShavingBeard_g23_c07.avi	ShavingBeard
520	v_TennisSwing_g14_c03.avi	TennisSwing
190	v_PlayingCello_g18_c07.avi	PlayingCello

Capture the frames of a video:

```
def crop_center_square(frame):
    y, x = frame.shape[0:2]
    min_dim = min(y, x)
    start_x = (x // 2) - (min_dim // 2)
    start_y = (y // 2) - (min_dim // 2)
    return frame[start_y : start_y + min_dim, start_x : start_x + min_dim]


def load_video(path, max_frames=0, resize=(IMG_SIZE, IMG_SIZE)):
    cap = cv2.VideoCapture(path)
    frames = []
    try:
        while True:
            ret, frame = cap.read()
            if not ret:
                break
```

```
            frame = crop_center_square(frame)
            frame = cv2.resize(frame, resize)
            frame = frame[:, :, [2, 1, 0]]
            frames.append(frame)

            if len(frames) == max_frames:
                break
    finally:
        cap.release()
    return np.array(frames)
```

Display the frames of a sample video:

```
video_path = "train/v_CricketShot_g08_c01.avi"
frames = load_video(video_path)
print("Video path :", video_path)
print("Frames shape:", frames.shape)

plt.figure(figsize=(10, 5))
plt.suptitle("v_CricketShot_g08_c01.avi  (CricketShot)")
for i in range(8):
    plt.subplot(2, 4, i + 1)
    plt.imshow(frames[i * 9])
    plt.title(f"Frame {i * 9}")
    plt.axis("off")
plt.show()
```

```
Video path : train/v_CricketShot_g08_c01.avi
Frames shape: (75, 224, 224, 3)
```



Build the InceptionV3 feature extractor (CNN):

```
def build_feature_extractor():
    feature_extractor = keras.applications.InceptionV3(
        weights="imagenet",
        include_top=False,
        pooling="avg",
        input_shape=(IMG_SIZE, IMG_SIZE, 3),
    )
    preprocess_input = keras.applications.inception_v3.preprocess_input

    inputs = keras.Input((IMG_SIZE, IMG_SIZE, 3))
    preprocessed = preprocess_input(inputs)

    outputs = feature_extractor(preprocessed)
    return keras.Model(inputs, outputs, name="feature_extractor")


feature_extractor = build_feature_extractor()
feature_extractor.summary()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/inception_v3/inception_v3_weights_tf_dim_ordering_tf_ke
87910968/87910968 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step

Model: "feature_extractor"
```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
true_divide (TrueDivide)	(None, 224, 224, 3)	0
subtract (Subtract)	(None, 224, 224, 3)	0
inception_v3 (Functional)	(None, 2048)	21,802,784

```
Total params: 21,802,784 (83.17 MB)
Trainable params: 21,768,352 (83.04 MB)
Non-trainable params: 34,432 (134.50 KB)
```

Encode the class labels:

```
label_processor = keras.layers.StringLookup(
    num_oov_indices=0, vocabulary=np.unique(train_df["tag"])
)
print(label_processor.get_vocabulary())
```

```
['CricketShot', 'PlayingCello', 'Punch', 'ShavingBeard', 'TennisSwing']
```

Extract the frame features of all videos:

```
def prepare_all_videos(df, root_dir):
    num_samples = len(df)
    video_paths = df["video_name"].values.tolist()
    labels = df["tag"].values
    labels = keras.ops.convert_to_numpy(label_processor(labels[..., None]))

    # `frame_masks` and `frame_features` are what we will feed to our sequence model.
    # `frame_masks` will contain a bunch of booleans denoting if a timestep is
    # masked with padding or not.
    frame_masks = np.zeros(shape=(num_samples, MAX_SEQ_LENGTH), dtype="bool")
    frame_features = np.zeros(
        shape=(num_samples, MAX_SEQ_LENGTH, NUM_FEATURES), dtype="float32"
    )

    # For each video.
    for idx, path in enumerate(video_paths):
        # Gather all its frames and add a batch dimension.
        frames = load_video(os.path.join(root_dir, path))
        frames = frames[None, ...]

        # Initialize placeholders to store the masks and features of the current video.
        temp_frame_mask = np.zeros(shape=(1, MAX_SEQ_LENGTH,), dtype="bool")
        temp_frame_features = np.zeros(
            shape=(1, MAX_SEQ_LENGTH, NUM_FEATURES), dtype="float32"
        )

        # Extract features from the frames of the current video.
        for i, batch in enumerate(frames):
            video_length = batch.shape[0]
            length = min(MAX_SEQ_LENGTH, video_length)
            for j in range(length):
                temp_frame_features[i, j, :] = feature_extractor.predict(
                    batch[None, j, :], verbose=0,
                )
            temp_frame_mask[i, :length] = 1  # 1 = not masked, 0 = masked

        frame_features[idx,] = temp_frame_features.squeeze()
        frame_masks[idx,] = temp_frame_mask.squeeze()

    return (frame_features, frame_masks), labels


train_data, train_labels = prepare_all_videos(train_df, "train")
test_data, test_labels = prepare_all_videos(test_df, "test")

print(f"Frame features in train set: {train_data[0].shape}")
print(f"Frame masks in train set: {train_data[1].shape}")
```

```
Frame features in train set: (594, 20, 2048)
Frame masks in train set: (594, 20)
```

Visualise the extracted features:

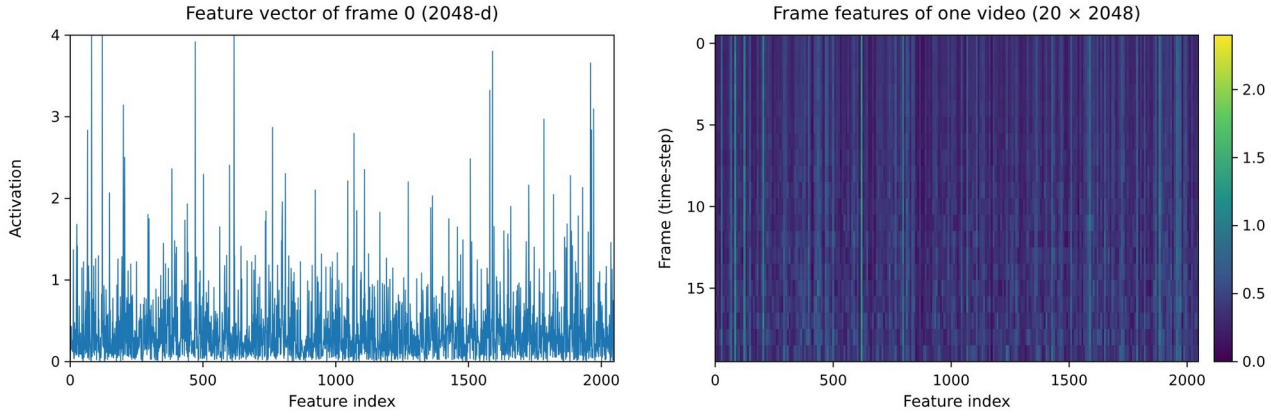
```
sample_features = train_data[0][0]          # (20, 2048) features of the first video
sample_mask = train_data[1][0]

print("Feature vector shape :", sample_features[0].shape)
print("Features of one video:", sample_features.shape)
print("Frame mask           :", sample_mask)

plt.figure(figsize=(14, 4))
plt.subplot(1, 2, 1)
plt.plot(sample_features[0])
plt.title("Feature vector of frame 0 (2048-d)")
plt.xlabel("Feature index"); plt.ylabel("Activation")

plt.subplot(1, 2, 2)
plt.imshow(sample_features, aspect="auto", cmap="viridis")
plt.colorbar()
plt.title("Frame features of one video (20 x 2048)")
plt.xlabel("Feature index"); plt.ylabel("Frame (time-step)")
plt.show()
```

```
Feature vector shape : (2048,)
Features of one video: (20, 2048)
Frame mask           : [ True  True  True  True  True  True  True  True  True  True  True  True
                         True  True  True  True  True  True  True  True]
```



RESULT:

Thus, the spatial features of the UCF101 video frames were successfully extracted using the pre-trained InceptionV3 CNN. Each video was converted into a (20, 2048) sequence of frame features with a frame mask, which is used by the RNN in the next experiments.

EX.NO.6	RNN/GRU Temporal Sequence Modeling
DATE: 17.09.26	

AIM:

To build and train a Recurrent Neural Network (stacked GRU layers) over the sequence of CNN frame features so that the temporal relationship between consecutive frames is learned and the video can be classified into its action class.

DESCRIPTION:

A video is an ordered sequence of frames. Each frame carries spatial information (extracted by the CNN in Experiment 5) and the order of the frames carries temporal information. A Recurrent Neural Network processes the frame features one time-step at a time and keeps a hidden state h(t) that summarises everything seen so far. A Gated Recurrent Unit (GRU) controls this hidden state with two gates:

Update gate : $z(t) = \text{sigmoid}(Wz \cdot x(t) + Uz \cdot h(t-1))$

Reset gate : $r(t) = \text{sigmoid}(Wr \cdot x(t) + Ur \cdot h(t-1))$

Candidate : $\tilde{h}(t) = \tanh(W \cdot x(t) + U \cdot (r(t) * h(t-1)))$

New state : $h(t) = (1 - z(t)) * h(t-1) + z(t) * \tilde{h}(t)$

ALGORITHM:

1. Take the frame_features (20 × 2048) and frame_masks produced by the CNN in Experiment 5 and the integer class labels.
2. Define the input layers: frame features (20, 2048) and the boolean mask (20,).
3. Add a GRU layer with 16 units (return_sequences = True) that uses the mask to skip padded frames.
4. Add a second GRU layer with 8 units to obtain one vector for the whole video.
5. Add Dropout (0.4), Dense (8, ReLU) and the output Dense layer with Softmax (5 classes).
6. Compile the model with the Adam optimizer, sparse categorical cross-entropy loss and the accuracy metric.
7. Train for 10 epochs with validation_split = 0.3 and save the best weights using ModelCheckpoint.
8. Reload the best weights, evaluate on the test videos and plot the training / validation curves.

ARCHITECTURE DIAGRAM:

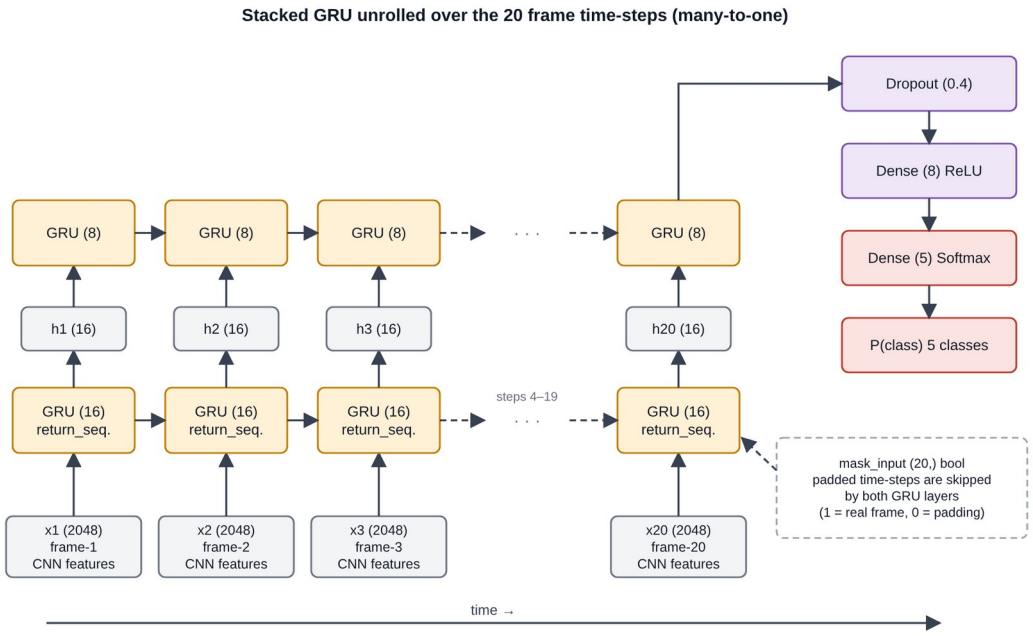


Fig. 6.1 Stacked GRU unrolled over the 20 time-steps of a video

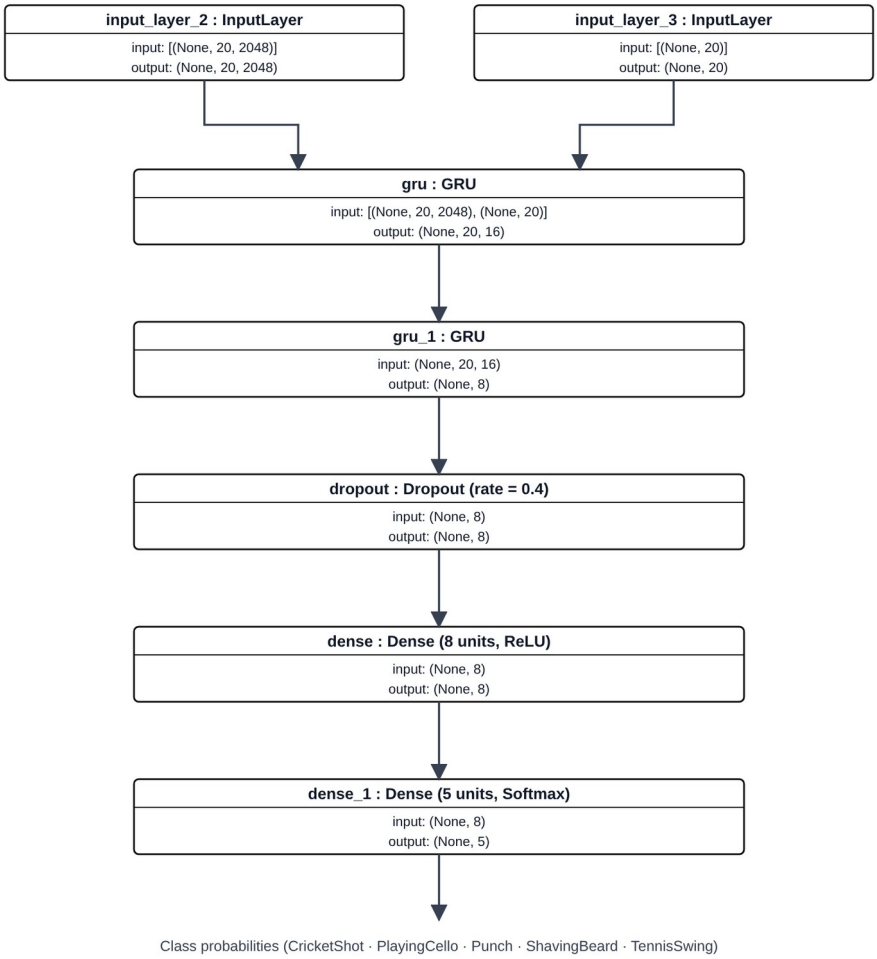


Fig. 6.2 Layer-wise architecture of the GRU sequence model

PROGRAM:

Define the sequence model and the experiment utility:

```
# Utility for our sequence model.
def get_sequence_model():
    class_vocab = label_processor.get_vocabulary()

    frame_features_input = keras.Input((MAX_SEQ_LENGTH, NUM_FEATURES))
    mask_input = keras.Input((MAX_SEQ_LENGTH,), dtype="bool")

    # Refer to the following tutorial to understand the significance of using `mask`:
    # https://keras.io/api/layers/recurrent_layers/gru/
    x = keras.layers.GRU(16, return_sequences=True)(
        frame_features_input, mask=mask_input
    )
    x = keras.layers.GRU(8)(x)
    x = keras.layers.Dropout(0.4)(x)
    x = keras.layers.Dense(8, activation="relu")(x)
    output = keras.layers.Dense(len(class_vocab), activation="softmax")(x)

    rnn_model = keras.Model([frame_features_input, mask_input], output)

    rnn_model.compile(
        loss="sparse_categorical_crossentropy", optimizer="adam", metrics=["accuracy"]
    )
    return rnn_model


# Utility for running experiments.
def run_experiment():
```

```
    filepath = "/tmp/video_classifier/ckpt.weights.h5"
    checkpoint = keras.callbacks.ModelCheckpoint(
        filepath, save_weights_only=True, save_best_only=True, verbose=1
    )

    seq_model = get_sequence_model()
    history = seq_model.fit(
        [train_data[0], train_data[1]],
        train_labels,
        validation_split=0.3,
        epochs=EPOCHS,
        callbacks=[checkpoint],
    )

    seq_model.load_weights(filepath)
    _, accuracy = seq_model.evaluate([test_data[0], test_data[1]], test_labels)
    print(f"Test accuracy: {round(accuracy * 100, 2)}%")

    return history, seq_model
```

Display the model summary:

```
get_sequence_model().summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer_2 (InputLayer)	(None, 20, 2048)	0	-
input_layer_3 (InputLayer)	(None, 20)	0	-
gru (GRU)	(None, 20, 16)	99,168	input_layer_2[0][0], input_layer_3[0][0]
gru_1 (GRU)	(None, 8)	624	gru[0][0]
dropout (Dropout)	(None, 8)	0	gru_1[0][0]
dense (Dense)	(None, 8)	72	dropout[0][0]
dense_1 (Dense)	(None, 5)	45	dense[0][0]

Total params: 99,909 (390.27 KB)
Trainable params: 99,909 (390.27 KB)
Non-trainable params: 0 (0.00 B)

Train the model and evaluate it on the test set:

```
history, sequence_model = run_experiment()
```

```
Epoch 1/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 88ms/step - accuracy: 0.2412 - loss: 1.6120
Epoch 1: val_loss improved from inf to 1.54120, saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 6s 163ms/step - accuracy: 0.2784 - loss: 1.5936 - val_accuracy: 0.4190 - val_loss: 1.5412
Epoch 2/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.4107 - loss: 1.4023
Epoch 2: val_loss improved from 1.54120 to 1.32740, saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.4619 - loss: 1.3612 - val_accuracy: 0.5642 - val_loss: 1.3274
Epoch 3/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.5315 - loss: 1.2216
Epoch 3: val_loss improved from 1.32740 to 1.16300, saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5593 - loss: 1.1904 - val_accuracy: 0.6369 - val_loss: 1.1630
Epoch 4/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.6102 - loss: 1.0602
Epoch 4: val_loss improved from 1.16300 to 1.03070, saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.6286 - loss: 1.0381 - val_accuracy: 0.6760 - val_loss: 1.0307
Epoch 5/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.6733 - loss: 0.9390
Epoch 5: val_loss improved from 1.03070 to 0.94060, saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.6842 - loss: 0.9172 - val_accuracy: 0.7095 - val_loss: 0.9406
Epoch 6/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.7061 - loss: 0.8402
Epoch 6: val_loss improved from 0.94060 to 0.86720, saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.7234 - loss: 0.8213 - val_accuracy: 0.7430 - val_loss: 0.8672
Epoch 7/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.7392 - loss: 0.7603
Epoch 7: val_loss did not improve from 0.86720
```

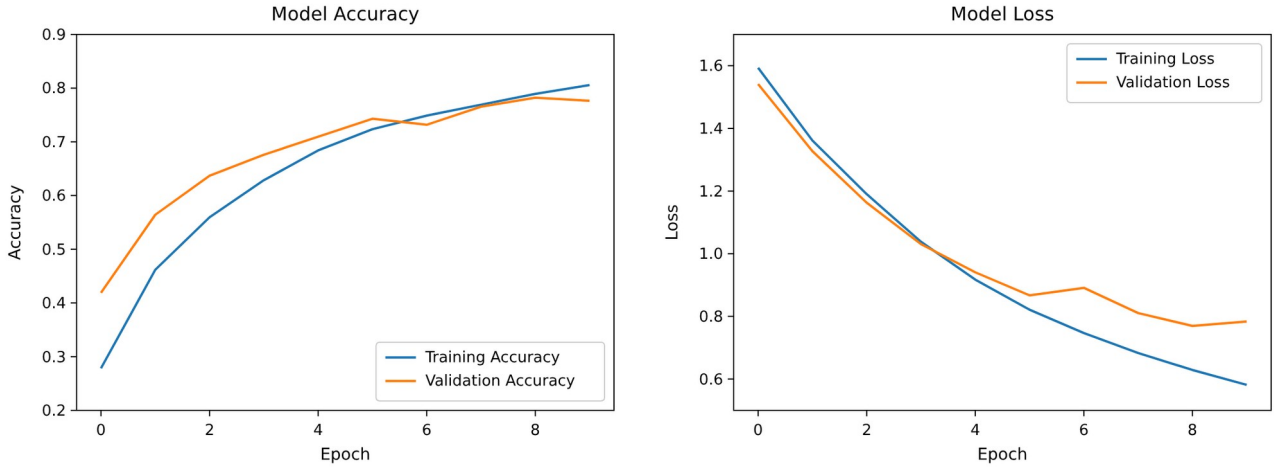
```
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.7488 - loss: 0.7466 - val_accuracy: 0.7318 - val_loss: 0.8913
Epoch 8/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.7625 - loss: 0.6932
Epoch 8: val_loss improved from 0.86720 to 0.81050, saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.7692 - loss: 0.6829 - val_accuracy: 0.7654 - val_loss: 0.8105
Epoch 9/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.7810 - loss: 0.6414
Epoch 9: val_loss improved from 0.81050 to 0.76980, saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.7894 - loss: 0.6291 - val_accuracy: 0.7821 - val_loss: 0.7698
Epoch 10/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.7977 - loss: 0.5903
Epoch 10: val_loss did not improve from 0.76980
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8057 - loss: 0.5815 - val_accuracy: 0.7765 - val_loss: 0.7835
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.7991 - loss: 0.7437
Test accuracy: 79.91%
```

Plot the accuracy and loss curves:

```
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Model Accuracy")
plt.xlabel("Epoch"); plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Model Loss")
plt.xlabel("Epoch"); plt.ylabel("Loss")
plt.legend()
plt.show()
```



RESULT:

Thus, the stacked GRU sequence model (99,909 trainable parameters) was trained on the CNN frame features. The model learned the temporal relationship between the frames and classified the UCF101 videos with a test accuracy of 79.91%.

EX.NO.7	End-to-End CNN+RNN Video Classification Pipeline
DATE: 24.09.26	

AIM:

To implement an end-to-end CNN-RNN video classification pipeline in which a pre-trained CNN (InceptionV3) extracts the spatial features of the video frames and a GRU based RNN learns the temporal pattern to predict the action performed in an unseen video.

ALGORITHM:

1. Select a test video from the UCF101 test set.
2. Read the video with OpenCV, crop the centre square, resize each frame to 224 × 224 and convert BGR to RGB.
3. Pass the first 20 frames through the InceptionV3 feature extractor to obtain a (1, 20, 2048) feature tensor and create the frame mask.
4. Feed the frame features and the mask to the trained GRU sequence model.
5. Sort the class probabilities with np.argsort() and print the predicted classes in descending order.
6. Convert the frames to an animated GIF to visualise the video and plot the class probabilities as a bar chart.

ARCHITECTURE DIAGRAM:

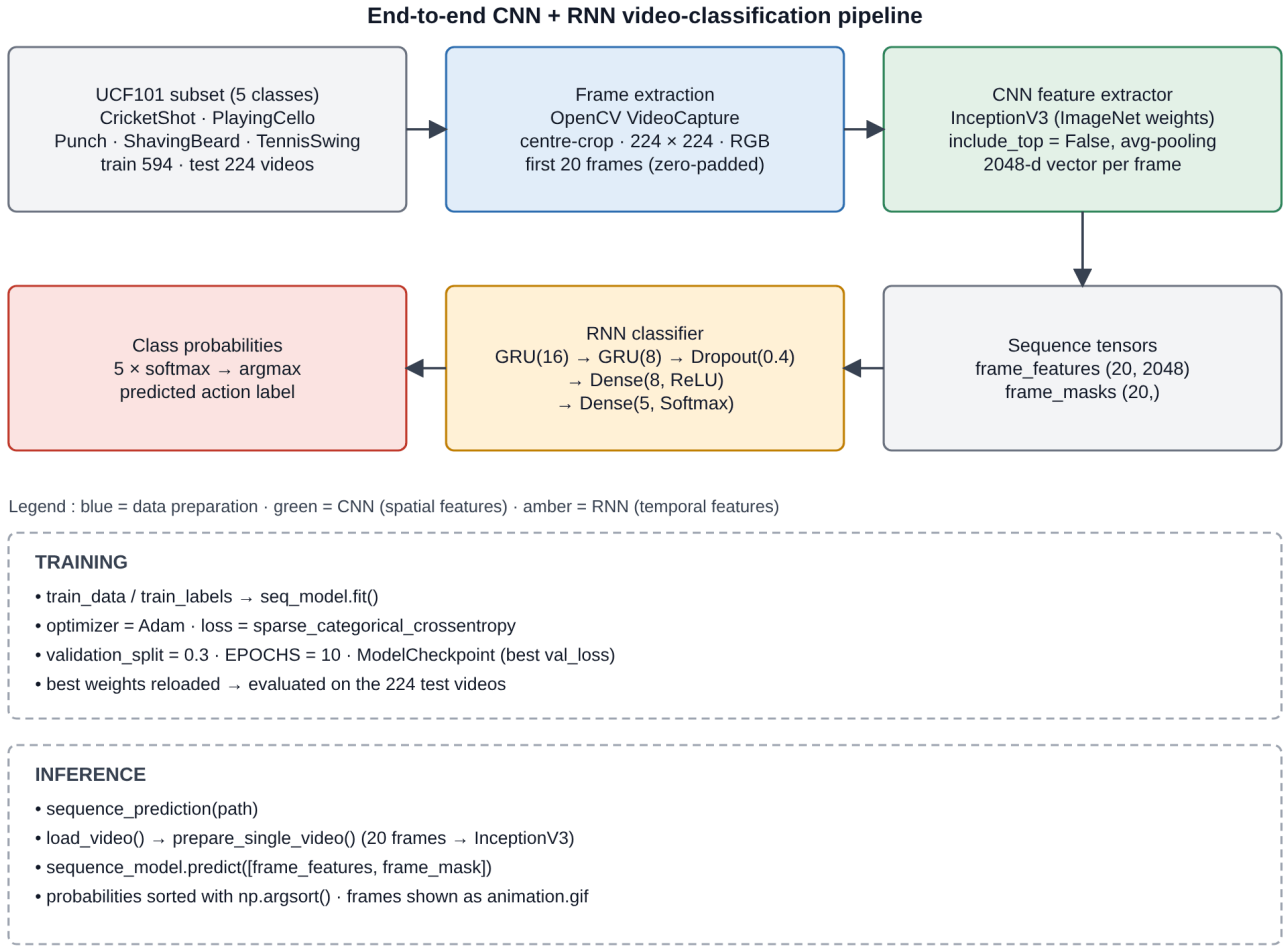


Fig. 7.1 End-to-end CNN + RNN video classification pipeline

PROGRAM:

Prepare a single video and predict its class:

```
def prepare_single_video(frames):
    frames = frames[None, ...]
    frame_mask = np.zeros(shape=(1, MAX_SEQ_LENGTH,), dtype="bool")
    frame_features = np.zeros(shape=(1, MAX_SEQ_LENGTH, NUM_FEATURES), dtype="float32")

    for i, batch in enumerate(frames):
        video_length = batch.shape[0]
        length = min(MAX_SEQ_LENGTH, video_length)
        for j in range(length):
            frame_features[i, j, :] = feature_extractor.predict(batch[None, j, :])
        frame_mask[i, :length] = 1  # 1 = not masked, 0 = masked

    return frame_features, frame_mask


def sequence_prediction(path):
    class_vocab = label_processor.get_vocabulary()

    frames = load_video(os.path.join("test", path))
    frame_features, frame_mask = prepare_single_video(frames)
    probabilities = sequence_model.predict([frame_features, frame_mask])[0]

    for i in np.argsort(probabilities)[::-1]:
        print(f"  {class_vocab[i]}: {probabilities[i] * 100:5.2f}%")
    return frames, probabilities


# This utility is for visualization.
# Referenced from:
# https://www.tensorflow.org/hub/tutorials/action_recognition_with_tf_hub
def to_gif(images):
    converted_images = images.astype(np.uint8)
    imageio.mimsave("animation.gif", converted_images, duration=100)
    return Image("animation.gif")
```

Run the pipeline on a test video:

```
test_video = "v_CricketShot_g01_c02.avi"
print(f"Test video path: {test_video}")
test_frames, probabilities = sequence_prediction(test_video)
to_gif(test_frames[:MAX_SEQ_LENGTH])
```

```
Test video path: v_CricketShot_g01_c02.avi
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 148ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 117ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 136ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 124ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 127ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 131ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 118ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 122ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 115ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 117ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 123ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 128ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
  CricketShot: 71.84%
  TennisSwing: 12.37%
```

```
  Punch:  7.51%
  ShavingBeard:  4.89%
  PlayingCello:  3.39%
```

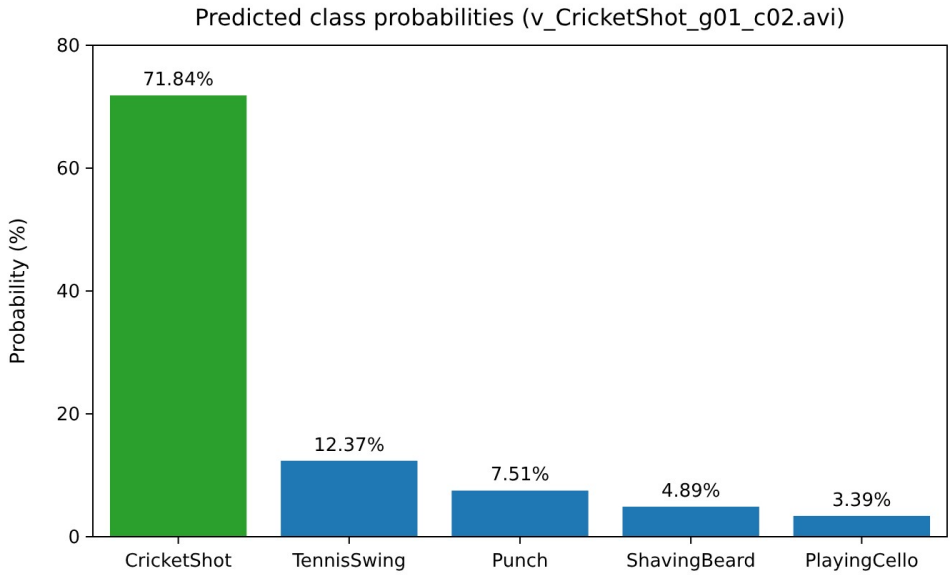
Frames of the test video (animation.gif):



Plot the predicted class probabilities:

```
class_vocab = label_processor.get_vocabulary()
order = np.argsort(probabilities)[::-1]

plt.figure(figsize=(7, 4.5))
bars = plt.bar([class_vocab[i] for i in order], probabilities[order] * 100)
bars[0].set_color("green")
for b, v in zip(bars, probabilities[order] * 100):
    plt.text(b.get_x() + b.get_width() / 2, v + 1, f"{v:.2f}%", ha="center")
plt.ylabel("Probability (%)")
plt.title("Predicted class probabilities  (" + test_video + ")")
plt.show()
```



RESULT:

Thus, the end-to-end CNN + RNN video classification pipeline was implemented successfully. InceptionV3 extracted the spatial features of the frames and the GRU network learned the temporal information; the test video v_CricketShot_g01_c02.avi was correctly classified as CricketShot with a probability of 71.84%.

EX.NO.8	Vanilla GAN for Image Generation
DATE: 24.09.26	

AIM:

To implement a Vanilla Generative Adversarial Network (GAN) with fully-connected Generator and Discriminator networks that learns to generate handwritten-digit images from random noise using the MNIST dataset.

ALGORITHM:

1. Load the MNIST dataset, scale the pixel values to [-1, 1] and flatten each image to a 784-dimensional vector.
2. Build the Generator that maps a 100-dimensional noise vector to a 784-dimensional image using Dense layers, LeakyReLU and a tanh output.
3. Build the Discriminator that classifies a 784-dimensional image as real or fake using Dense layers, LeakyReLU, Dropout and a sigmoid output.
4. Define the binary cross-entropy loss for both networks and the Adam optimizers (learning rate 0.0002, beta_1 = 0.5).
5. For every batch: generate fake images from noise, train the Discriminator on real images (label 1) and fake images (label 0).
6. Train the Generator so that the Discriminator classifies the generated images as real (label 1).
7. Repeat the training for 50 epochs and record the Generator and Discriminator losses.
8. Generate images from a fixed noise vector at selected epochs and plot the loss curves.

ARCHITECTURE DIAGRAM:

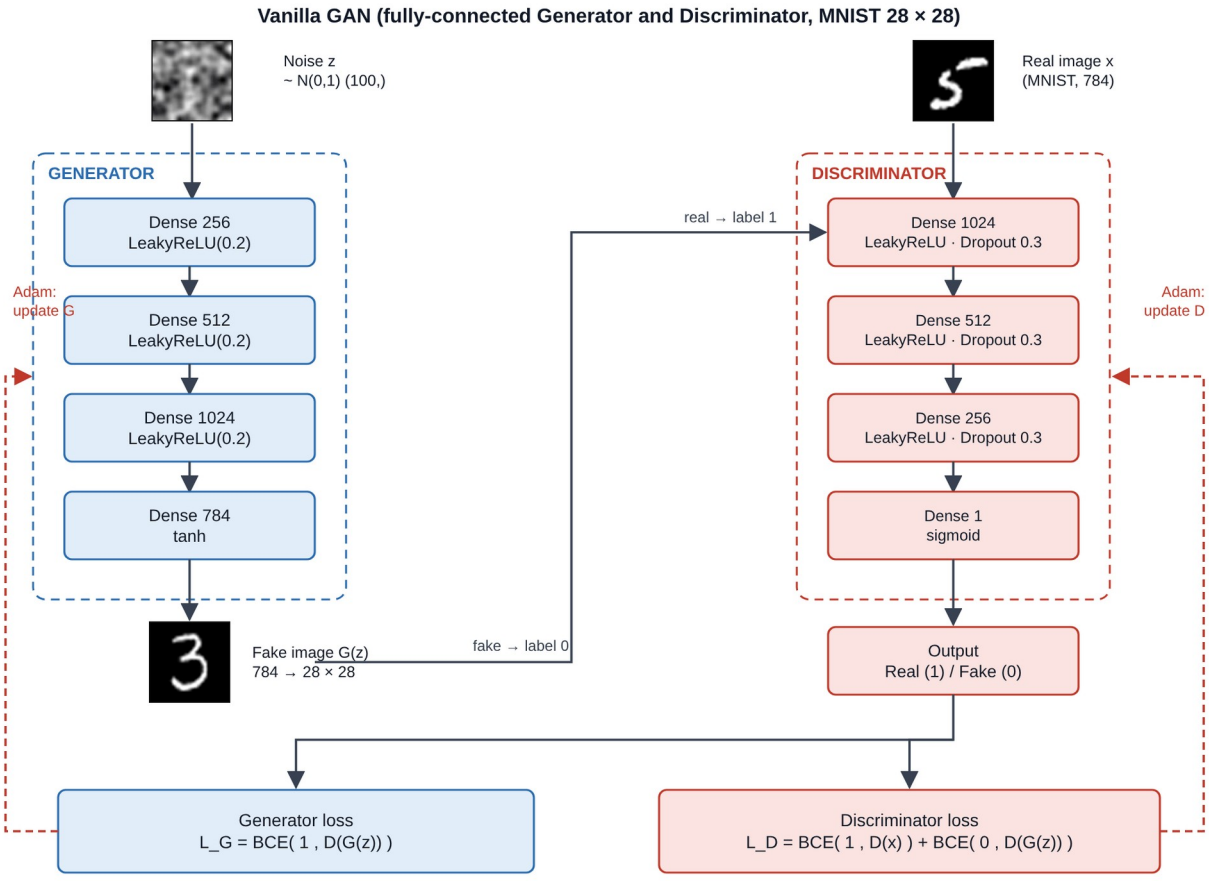


Fig. 8.1 Architecture of the Vanilla GAN

PROGRAM:

Import libraries and load the dataset:

```
import time
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt

(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
x_train = (x_train.astype("float32") - 127.5) / 127.5    # scale to [-1, 1]
x_train = x_train.reshape(-1, 784)

BUFFER_SIZE = 60000
BATCH_SIZE = 256
EPOCHS = 50
noise_dim = 100

train_dataset = tf.data.Dataset.from_tensor_slices(x_train).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

print("Training data shape :", x_train.shape)
print("Pixel range         :", x_train.min(), x_train.max())
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training data shape : (60000, 784)
Pixel range         : -1.0 1.0
```

Build the Generator and the Discriminator:

```
def make_generator_model():
    model = tf.keras.Sequential([
        layers.Input(shape=(noise_dim,)),
        layers.Dense(256), layers.LeakyReLU(0.2),
        layers.Dense(512), layers.LeakyReLU(0.2),
        layers.Dense(1024), layers.LeakyReLU(0.2),
        layers.Dense(784, activation="tanh"),
    ])
    return model


def make_discriminator_model():
    model = tf.keras.Sequential([
        layers.Input(shape=(784,)),
        layers.Dense(1024), layers.LeakyReLU(0.2), layers.Dropout(0.3),
        layers.Dense(512), layers.LeakyReLU(0.2), layers.Dropout(0.3),
        layers.Dense(256), layers.LeakyReLU(0.2), layers.Dropout(0.3),
        layers.Dense(1, activation="sigmoid"),
    ])
    return model


generator = make_generator_model()
discriminator = make_discriminator_model()
generator.summary()
discriminator.summary()
```

GENERATOR

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	25,856
leaky_re_lu (LeakyReLU)	(None, 256)	0
dense_1 (Dense)	(None, 512)	131,584
leaky_re_lu_1 (LeakyReLU)	(None, 512)	0
dense_2 (Dense)	(None, 1024)	525,312
leaky_re_lu_2 (LeakyReLU)	(None, 1024)	0
dense_3 (Dense)	(None, 784)	803,600

Total params: 1,486,352 (5.67 MB)
Trainable params: 1,486,352 (5.67 MB)
Non-trainable params: 0 (0.00 B)

DISCRIMINATOR

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 1024)	803,840
leaky_re_lu_3 (LeakyReLU)	(None, 1024)	0
dropout (Dropout)	(None, 1024)	0
dense_5 (Dense)	(None, 512)	524,800
leaky_re_lu_4 (LeakyReLU)	(None, 512)	0
dropout_1 (Dropout)	(None, 512)	0
dense_6 (Dense)	(None, 256)	131,328
leaky_re_lu_5 (LeakyReLU)	(None, 256)	0
dropout_2 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 1)	257

Total params: 1,460,225 (5.57 MB)
Trainable params: 1,460,225 (5.57 MB)
Non-trainable params: 0 (0.00 B)

Define the loss functions, optimizers and the training step:

```
cross_entropy = tf.keras.losses.BinaryCrossentropy()

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

generator_optimizer = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)
discriminator_optimizer = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)

@tf.function
def train_step(real_images):
    noise = tf.random.normal([tf.shape(real_images)[0], noise_dim])
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        fake_images = generator(noise, training=True)
        real_output = discriminator(real_images, training=True)
        fake_output = discriminator(fake_images, training=True)
        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gen_grads = gen_tape.gradient(gen_loss, generator.trainable_variables)
    disc_grads = disc_tape.gradient(disc_loss, discriminator.trainable_variables)
    generator_optimizer.apply_gradients(zip(gen_grads, generator.trainable_variables))
```

```
    discriminator_optimizer.apply_gradients(zip(disc_grads, discriminator.trainable_variables))
    return disc_loss, gen_loss
```

Train the GAN:

```
seed = tf.random.normal([16, noise_dim])        # fixed noise to monitor progress
d_history, g_history = [], []

def generate_and_save_images(model, epoch, test_input):
    predictions = model(test_input, training=False).numpy().reshape(-1, 28, 28)
    plt.figure(figsize=(4, 4))
    for i in range(16):
        plt.subplot(4, 4, i + 1)
        plt.imshow(predictions[i] * 127.5 + 127.5, cmap="gray")
        plt.axis("off")
    plt.savefig(f"vanilla_gan_epoch_{epoch:04d}.png")
    plt.close()

def train(dataset, epochs):
    print("Training started...")
    for epoch in range(1, epochs + 1):
        start = time.time()
        d_losses, g_losses = [], []
        for batch in dataset:
            d_l, g_l = train_step(batch)
            d_losses.append(d_l.numpy()); g_losses.append(g_l.numpy())
        d_history.append(float(np.mean(d_losses)))
        g_history.append(float(np.mean(g_losses)))

        if epoch in (1, 10, 30, 50):
            generate_and_save_images(generator, epoch, seed)
        if epoch == 1 or epoch % 5 == 0:
            print(f"Epoch {epoch:2d}/{epochs} | D loss: {d_history[-1]:.4f} | "
                  f"G loss: {g_history[-1]:.4f} | time: {time.time() - start:.2f} s")
    print("\nTraining completed successfully.")

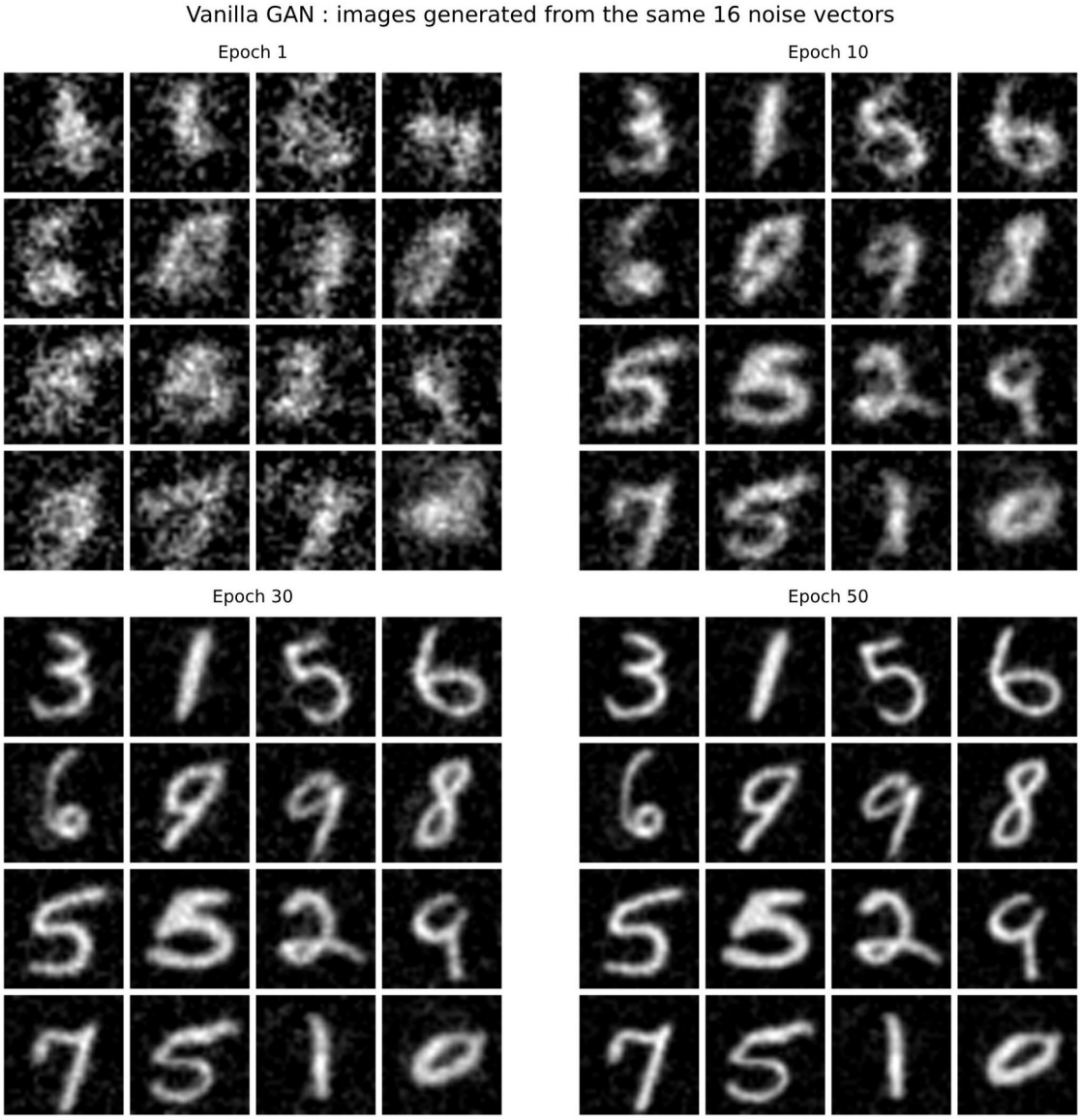
train(train_dataset, EPOCHS)
```

```
Training started...
Epoch  1/50 | D loss: 0.7560 | G loss: 1.8079 | time: 6.84 s
Epoch  5/50 | D loss: 1.0756 | G loss: 1.4440 | time: 3.06 s
Epoch 10/50 | D loss: 1.1533 | G loss: 1.1055 | time: 3.08 s
Epoch 15/50 | D loss: 1.1502 | G loss: 1.0691 | time: 3.10 s
Epoch 20/50 | D loss: 1.1612 | G loss: 0.9875 | time: 3.12 s
Epoch 25/50 | D loss: 1.0884 | G loss: 0.9857 | time: 3.14 s
Epoch 30/50 | D loss: 1.1765 | G loss: 0.9768 | time: 3.16 s
Epoch 35/50 | D loss: 1.1894 | G loss: 1.0713 | time: 3.18 s
Epoch 40/50 | D loss: 1.1733 | G loss: 1.0617 | time: 3.20 s
Epoch 45/50 | D loss: 1.1671 | G loss: 1.0749 | time: 3.05 s
Epoch 50/50 | D loss: 1.1967 | G loss: 1.1709 | time: 3.07 s

Training completed successfully.
```

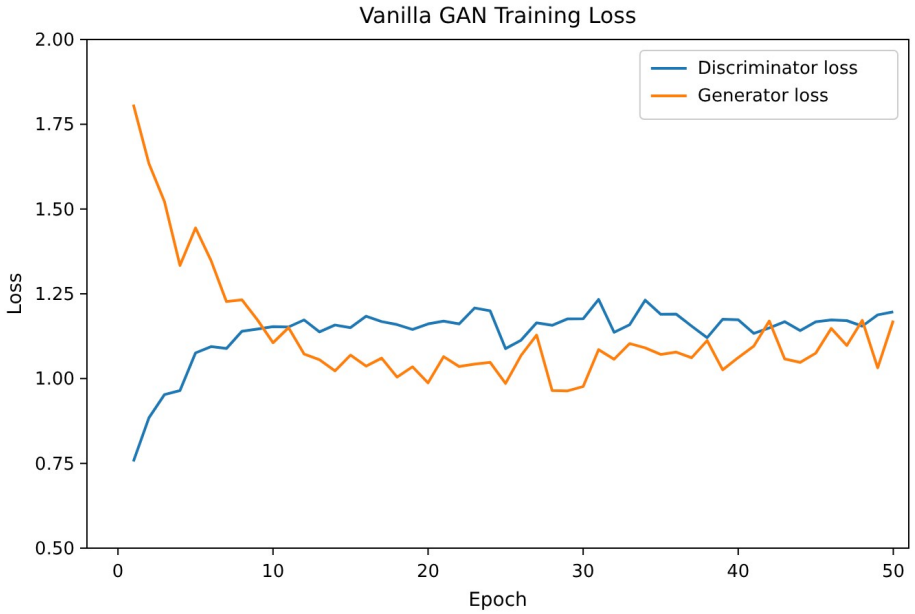
Display the generated images at different epochs:

```
plt.figure(figsize=(10, 10))
plt.suptitle("Vanilla GAN : images generated from the same 16 noise vectors")
for k, ep in enumerate([1, 10, 30, 50]):
    plt.subplot(2, 2, k + 1)
    plt.imshow(plt.imread(f"vanilla_gan_epoch_{ep:04d}.png"))
    plt.title(f"Epoch {ep}")
    plt.axis("off")
plt.show()
```



Plot the training losses:

```
plt.figure(figsize=(7, 4.6))
plt.plot(range(1, EPOCHS + 1), d_history, label="Discriminator loss")
plt.plot(range(1, EPOCHS + 1), g_history, label="Generator loss")
plt.title("Vanilla GAN Training Loss")
plt.xlabel("Epoch"); plt.ylabel("Loss")
plt.legend()
plt.show()
```



RESULT:

Thus, the Vanilla GAN was implemented successfully. After 50 epochs the Generator produced recognisable handwritten digits from random noise while the Discriminator and Generator losses settled around 1.1, showing that the two networks reached a balance.

EX.NO.9	DCGAN (Deep Convolutional GAN)
DATE: 24.09.26	

AIM:

To implement a Deep Convolutional GAN (DCGAN) in which the Generator uses transposed convolutions and the Discriminator uses strided convolutions to generate handwritten-digit images from random noise using the MNIST dataset.

ALGORITHM:

1. Load the MNIST dataset, reshape the images to (28, 28, 1), normalise them to [-1, 1] and create shuffled batches of 256 images.
2. Build the Generator: Dense (7·7·256) → BatchNorm → LeakyReLU → Reshape → three Conv2DTranspose layers (128, 64 and 1 filters) with a tanh output.
3. Build the Discriminator: two Conv2D layers (64 and 128 filters, stride 2) with LeakyReLU and Dropout, followed by Flatten and Dense (1).
4. Define the cross-entropy losses and the Adam optimizers (learning rate 0.0001).
5. In every training step generate images from noise, obtain the Discriminator outputs for real and fake images and compute both losses.
6. Update the Generator and the Discriminator using the gradients of their losses.
7. Repeat the training for 50 epochs and save the images generated from a fixed seed after selected epochs.
8. Display the generated images to observe the improvement of the Generator.

ARCHITECTURE DIAGRAM:

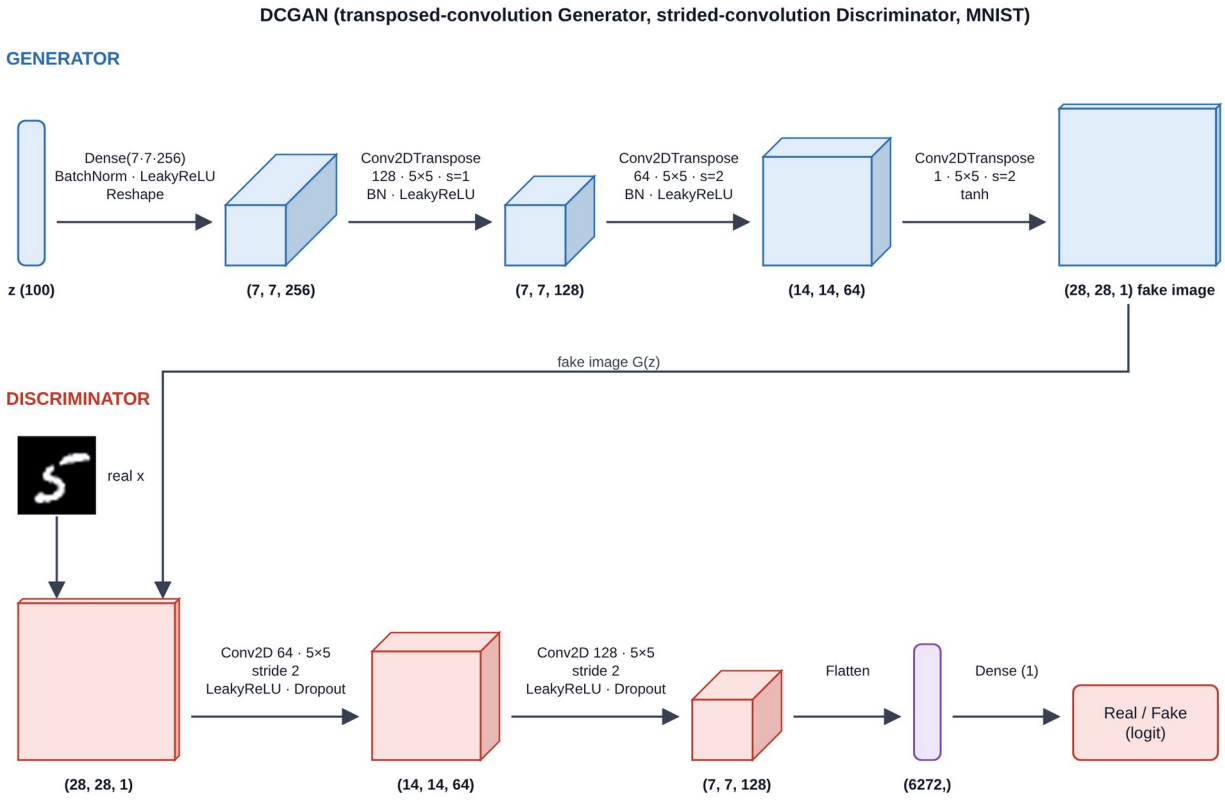


Fig. 9.1 Architecture of the DCGAN (Generator and Discriminator)

PROGRAM:

Import libraries and load the dataset:

```
import time
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt

(train_images, train_labels), (_, _) = tf.keras.datasets.mnist.load_data()
train_images = train_images.reshape(train_images.shape[0], 28, 28, 1).astype("float32")
train_images = (train_images - 127.5) / 127.5      # normalize the images to [-1, 1]

BUFFER_SIZE = 60000
BATCH_SIZE = 256
EPOCHS = 50
noise_dim = 100
num_examples_to_generate = 16

train_dataset = tf.data.Dataset.from_tensor_slices(train_images).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

print("Training images shape:", train_images.shape)
print("Pixel range           :", train_images.min(), train_images.max())
print("Number of batches     :", len(train_dataset), " (BATCH_SIZE = 256)")
```

```
Training images shape: (60000, 28, 28, 1)
Pixel range           : -1.0 1.0
Number of batches     : 235  (BATCH_SIZE = 256)
```

Create the Generator and test it with a noise vector:

```
def make_generator_model():
    model = tf.keras.Sequential()
    model.add(layers.Dense(7*7*256, use_bias=False, input_shape=(100,)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Reshape((7, 7, 256)))
    assert model.output_shape == (None, 7, 7, 256)

    model.add(layers.Conv2DTranspose(128, (5, 5), strides=(1, 1), padding="same", use_bias=False))
    assert model.output_shape == (None, 7, 7, 128)
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding="same", use_bias=False))
    assert model.output_shape == (None, 14, 14, 64)
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding="same",
                                     use_bias=False, activation="tanh"))
    assert model.output_shape == (None, 28, 28, 1)
    return model


generator = make_generator_model()
noise = tf.random.normal([1, 100])
generated_image = generator(noise, training=False)

plt.imshow(generated_image[0, :, :, 0], cmap="gray")
plt.title("Image generated by the untrained Generator")
plt.show()
```

Create the Discriminator and test it:

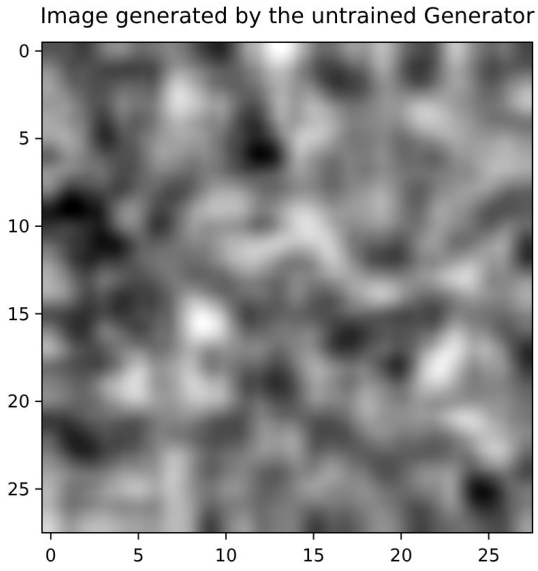
```
def make_discriminator_model():
    model = tf.keras.Sequential()
    model.add(layers.Conv2D(64, (5, 5), strides=(2, 2), padding="same", input_shape=[28, 28, 1]))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(128, (5, 5), strides=(2, 2), padding="same"))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
```

```
    model.add(layers.Dense(1))
    return model


discriminator = make_discriminator_model()
decision = discriminator(generated_image)
print("Discriminator decision (untrained):")
print(decision)
```




```
Discriminator decision (untrained):
tf.Tensor([[-0.00158323]],
 shape=(1, 1), dtype=float32)
```

Display the model summaries:

```
generator.summary()
discriminator.summary()
```

GENERATOR

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 12544)	1,254,400
batch_normalization (BatchNormalization)	(None, 12544)	50,176
leaky_re_lu (LeakyReLU)	(None, 12544)	0
reshape (Reshape)	(None, 7, 7, 256)	0
conv2d_transpose (Conv2DTranspose)	(None, 7, 7, 128)	819,200
batch_normalization_1 (BatchNormalization)	(None, 7, 7, 128)	512
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 14, 14, 64)	204,800
batch_normalization_2 (BatchNormalization)	(None, 14, 14, 64)	256
leaky_re_lu_2 (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	1,600

Total params: 2,330,944 (8.89 MB)
Trainable params: 2,305,472 (8.79 MB)
Non-trainable params: 25,472 (99.50 KB)

DISCRIMINATOR

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 14, 14, 64)	1,664
leaky_re_lu_3 (LeakyReLU)	(None, 14, 14, 64)	0
dropout (Dropout)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	204,928
leaky_re_lu_4 (LeakyReLU)	(None, 7, 7, 128)	0
dropout_1 (Dropout)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense_1 (Dense)	(None, 1)	6,273

Total params: 212,865 (831.50 KB)
Trainable params: 212,865 (831.50 KB)
Non-trainable params: 0 (0.00 B)

Define the loss functions and optimizers:

```
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)
```

Define the training loop:

```
seed = tf.random.normal([num_examples_to_generate, noise_dim])

@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, noise_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainable_variables))
    return gen_loss, disc_loss


def generate_and_save_images(model, epoch, test_input):
    predictions = model(test_input, training=False)
    fig = plt.figure(figsize=(4, 4))
    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i + 1)
        plt.imshow(predictions[i, :, :, 0] * 127.5 + 127.5, cmap="gray")
        plt.axis("off")
    plt.savefig("image_at_epoch_{:04d}.png".format(epoch))
    plt.close()


def train(dataset, epochs):
    print("Training started...")
    for epoch in range(1, epochs + 1):
```

```
        start = time.time()
        g_l, d_l = [], []
        for image_batch in dataset:
            gl, dl = train_step(image_batch)
            g_l.append(gl.numpy()); d_l.append(dl.numpy())

        if epoch in (1, 5, 10, 25, 50):
            generate_and_save_images(generator, epoch, seed)
        if epoch == 1 or epoch % 5 == 0:
            print(f"Epoch {epoch:2d}/{epochs} | gen_loss: {np.mean(g_l):.4f} | "
                  f"disc_loss: {np.mean(d_l):.4f} | time: {time.time() - start:.2f} s")
    print("\nTraining completed successfully.")
```

Train the model:

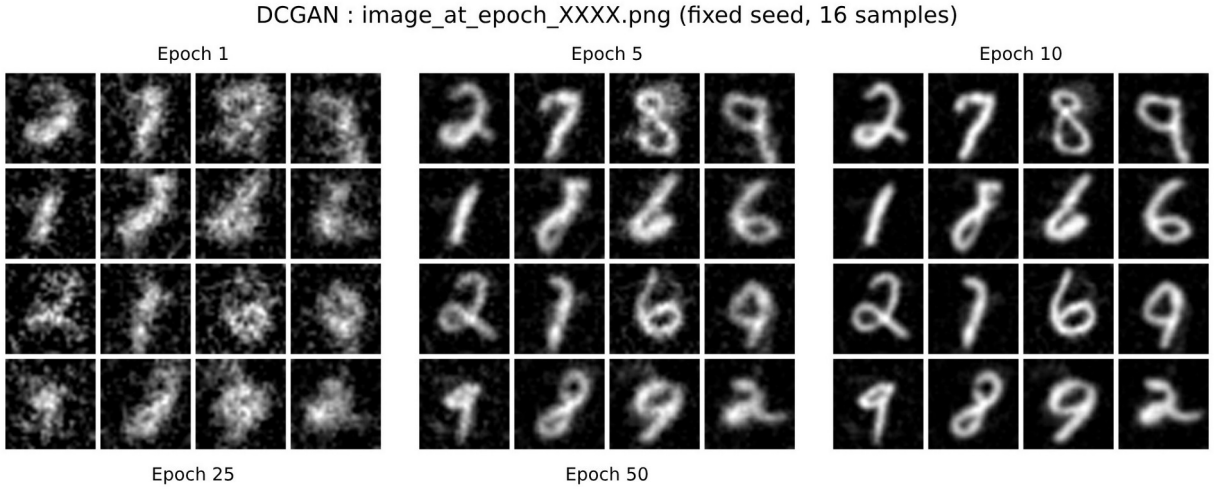
```
train(train_dataset, EPOCHS)
```

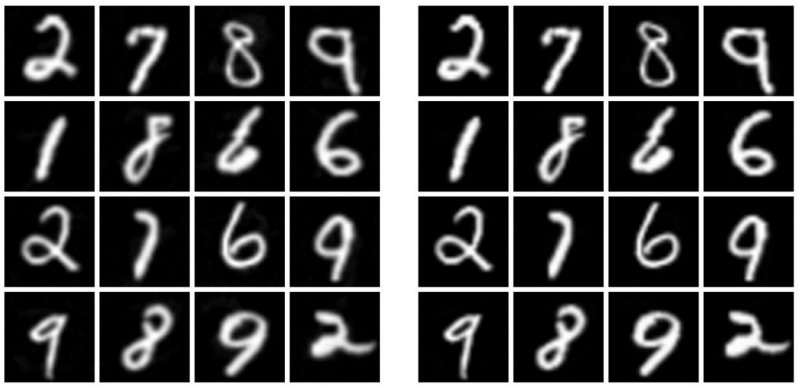
```
Training started...
Epoch  1/50 | gen_loss: 1.2653 | disc_loss: 1.1581 | time: 21.84 s
Epoch  5/50 | gen_loss: 1.0107 | disc_loss: 1.1361 | time: 9.35 s
Epoch 10/50 | gen_loss: 1.0319 | disc_loss: 1.3525 | time: 9.37 s
Epoch 15/50 | gen_loss: 0.8624 | disc_loss: 1.1726 | time: 9.39 s
Epoch 20/50 | gen_loss: 0.8844 | disc_loss: 1.2173 | time: 9.41 s
Epoch 25/50 | gen_loss: 1.0408 | disc_loss: 1.2204 | time: 9.43 s
Epoch 30/50 | gen_loss: 0.7848 | disc_loss: 1.2746 | time: 9.45 s
Epoch 35/50 | gen_loss: 0.9438 | disc_loss: 1.2339 | time: 9.47 s
Epoch 40/50 | gen_loss: 0.9163 | disc_loss: 1.1548 | time: 9.49 s
Epoch 45/50 | gen_loss: 0.8744 | disc_loss: 1.2312 | time: 9.34 s
Epoch 50/50 | gen_loss: 0.8947 | disc_loss: 1.2800 | time: 9.36 s

Training completed successfully.
```

Display the images generated during training:

```
plt.figure(figsize=(12, 8))
plt.suptitle("DCGAN : image_at_epoch_XXXX.png  (fixed seed, 16 samples)")
for k, ep in enumerate([1, 5, 10, 25, 50]):
    plt.subplot(2, 3, k + 1)
    plt.imshow(plt.imread("image_at_epoch_{:04d}.png".format(ep)))
    plt.title(f"Epoch {ep}")
    plt.axis("off")
plt.show()
```



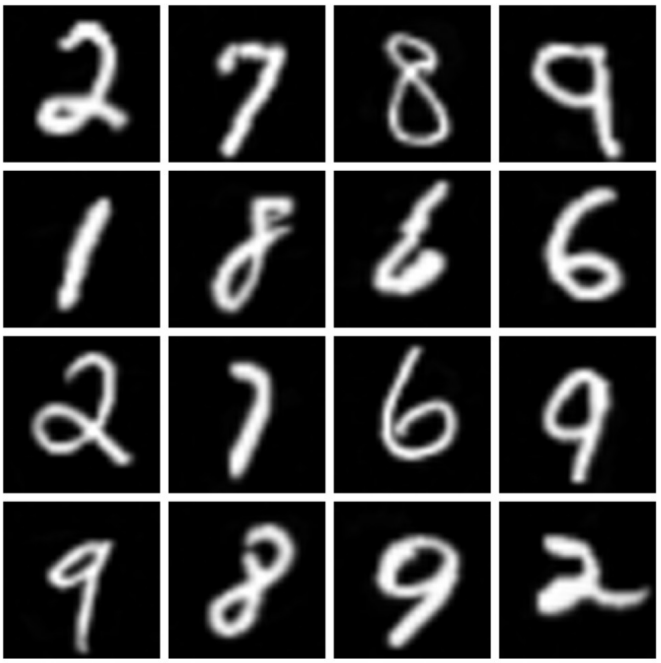


Display the final generated images:

```
def display_image(epoch_no):
    return plt.imshow(plt.imread("image_at_epoch_{:04d}.png".format(epoch_no)))

display_image(EPOCHS)
plt.axis("off")
plt.show()
```

display_image(EPOCHS) → image_at_epoch_0050.png



RESULT:

Thus, the DCGAN was implemented successfully. The Generator, built with transposed convolutions, learned to produce realistic handwritten digits from random noise, and the quality of the generated images improved steadily from the first epoch to the 50th epoch.

EX.NO.10	Conditional GAN (cGAN)
DATE: 24.09.26	

AIM:

To implement a Conditional GAN (cGAN) in which the class label is given to both the Generator and the Discriminator along with the random noise, so that images of a required class (digit 0 – 9) can be generated.

ALGORITHM:

1. Load the MNIST dataset (70,000 images), scale the pixels to [0, 1] and convert the labels to one-hot vectors.
2. Create a tf.data pipeline with a batch size of 64.
3. Build the Generator whose input is the 128-dimensional noise vector concatenated with the 10-dimensional one-hot label (138 values).
4. Build the Discriminator whose input is the image concatenated with the label maps (28 × 28 × (1 + 10) channels).
5. Define the ConditionalGAN model and its train_step(): train the Discriminator on real and generated images with their labels, then train the Generator to fool it.
6. Compile with Adam optimizers (learning rate 0.0003) and binary cross-entropy loss, then train for 20 epochs.
7. Use the trained Generator to interpolate between two classes (2 → 6) and to generate images for every class label.

ARCHITECTURE DIAGRAM:

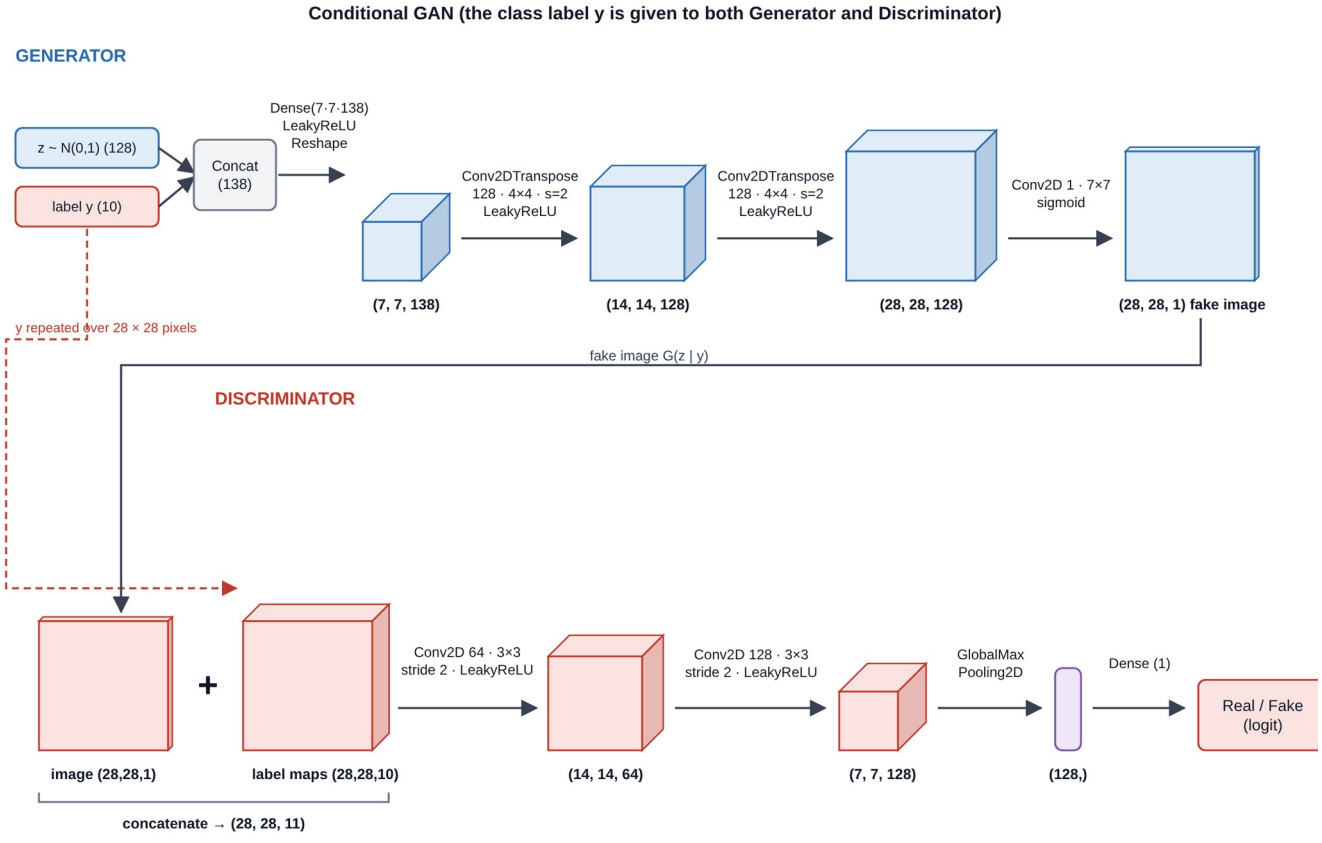


Fig. 10.1 Architecture of the Conditional GAN

PROGRAM:

Import libraries and set the hyper-parameters:

```
import keras
from keras import layers
from keras import ops
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import imageio

batch_size = 64
num_channels = 1
num_classes = 10
image_size = 28
latent_dim = 128
```

Load and pre-process the MNIST dataset:

```
# We'll use all the available examples from both the training and test sets.
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
all_digits = np.concatenate([x_train, x_test])
all_labels = np.concatenate([y_train, y_test])

# Scale the pixel values to [0, 1] range, add a channel dimension to
# the images, and one-hot encode the labels.
all_digits = all_digits.astype("float32") / 255.0
all_digits = np.reshape(all_digits, (-1, 28, 28, 1))
all_labels = keras.utils.to_categorical(all_labels, 10)

# Create tf.data.Dataset.
dataset = tf.data.Dataset.from_tensor_slices((all_digits, all_labels))
dataset = dataset.shuffle(buffer_size=1024).batch(batch_size)

print(f"Shape of training images: {all_digits.shape}")
print(f"Shape of training labels: {all_labels.shape}")
```

Calculate the number of input channels:

```
generator_in_channels = latent_dim + num_classes
discriminator_in_channels = num_channels + num_classes
print(generator_in_channels, discriminator_in_channels)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Shape of training images: (70000, 28, 28, 1)
Shape of training labels: (70000, 10)
138 11
```

Create the discriminator and the generator:

```
# Create the discriminator.
discriminator = keras.Sequential(
    [
        keras.layers.InputLayer((28, 28, discriminator_in_channels)),
        layers.Conv2D(64, (3, 3), strides=(2, 2), padding="same"),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Conv2D(128, (3, 3), strides=(2, 2), padding="same"),
        layers.LeakyReLU(negative_slope=0.2),
        layers.GlobalMaxPooling2D(),
        layers.Dense(1),
    ],
    name="discriminator",
)

# Create the generator.
generator = keras.Sequential(
    [
        keras.layers.InputLayer((generator_in_channels,)),
        # We want to generate 128 + num_classes coefficients to reshape into a
        # 7x7x(128 + num_classes) map.
        layers.Dense(7 * 7 * generator_in_channels),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Reshape((7, 7, generator_in_channels)),
        layers.Conv2DTranspose(128, (4, 4), strides=(2, 2), padding="same"),
        layers.LeakyReLU(negative_slope=0.2),
```

```
        layers.Conv2DTranspose(128, (4, 4), strides=(2, 2), padding="same"),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Conv2D(1, (7, 7), padding="same", activation="sigmoid"),
    ],
    name="generator",
)

discriminator.summary()
generator.summary()
```

DISCRIMINATOR

Model: "discriminator"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 14, 14, 64)	6,400
leaky_re_lu (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	73,856
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	0
global_max_pooling2d (GlobalMaxPooling2D)	(None, 128)	0
dense (Dense)	(None, 1)	129

Total params: 80,385 (314.00 KB)
Trainable params: 80,385 (314.00 KB)
Non-trainable params: 0 (0.00 B)

GENERATOR

Model: "generator"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 6762)	939,918
leaky_re_lu_2 (LeakyReLU)	(None, 6762)	0
reshape (Reshape)	(None, 7, 7, 138)	0
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 128)	282,752
leaky_re_lu_3 (LeakyReLU)	(None, 14, 14, 128)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 128)	262,272
leaky_re_lu_4 (LeakyReLU)	(None, 28, 28, 128)	0
conv2d_2 (Conv2D)	(None, 28, 28, 1)	6,273

Total params: 1,491,215 (5.69 MB)
Trainable params: 1,491,215 (5.69 MB)
Non-trainable params: 0 (0.00 B)

Create the ConditionalGAN model:

```
class ConditionalGAN(keras.Model):
    def __init__(self, discriminator, generator, latent_dim):
        super().__init__()
        self.discriminator = discriminator
        self.generator = generator
        self.latent_dim = latent_dim
        self.seed_generator = keras.random.SeedGenerator(1337)
        self.gen_loss_tracker = keras.metrics.Mean(name="generator_loss")
        self.disc_loss_tracker = keras.metrics.Mean(name="discriminator_loss")

    @property
    def metrics(self):
        return [self.gen_loss_tracker, self.disc_loss_tracker]

    def compile(self, d_optimizer, g_optimizer, loss_fn):
        super().compile()
        self.d_optimizer = d_optimizer
        self.g_optimizer = g_optimizer
```

```
        self.loss_fn = loss_fn

    def train_step(self, data):
        # Unpack the data.
        real_images, one_hot_labels = data

        # Add dummy dimensions to the labels so that they can be concatenated with
        # the images. This is for the discriminator.
        image_one_hot_labels = one_hot_labels[:, :, None, None]
        image_one_hot_labels = ops.repeat(
            image_one_hot_labels, repeats=[image_size * image_size]
        )
        image_one_hot_labels = ops.reshape(
            image_one_hot_labels, (-1, image_size, image_size, num_classes)
        )

        # Sample random points in the latent space and concatenate the labels.
        # This is for the generator.
        batch_size = ops.shape(real_images)[0]
        random_latent_vectors = keras.random.normal(
            shape=(batch_size, self.latent_dim), seed=self.seed_generator
        )
        random_vector_labels = ops.concatenate(
            [random_latent_vectors, one_hot_labels], axis=1
        )

        # Decode the noise (guided by labels) to fake images.
        generated_images = self.generator(random_vector_labels)

        # Combine them with real images. Note that we are concatenating the labels
        # with these images here.
        fake_image_and_labels = ops.concatenate(
            [generated_images, image_one_hot_labels], -1
        )
        real_image_and_labels = ops.concatenate([real_images, image_one_hot_labels], -1)
        combined_images = ops.concatenate(
            [fake_image_and_labels, real_image_and_labels], axis=0
        )

        # Assemble labels discriminating real from fake images.
        labels = ops.concatenate(
            [ops.ones((batch_size, 1)), ops.zeros((batch_size, 1))], axis=0
        )

        # Train the discriminator.
        with tf.GradientTape() as tape:
            predictions = self.discriminator(combined_images)
            d_loss = self.loss_fn(labels, predictions)
        grads = tape.gradient(d_loss, self.discriminator.trainable_weights)
        self.d_optimizer.apply_gradients(
            zip(grads, self.discriminator.trainable_weights)
        )

        # Sample random points in the latent space.
        random_latent_vectors = keras.random.normal(
            shape=(batch_size, self.latent_dim), seed=self.seed_generator
        )
        random_vector_labels = ops.concatenate(
            [random_latent_vectors, one_hot_labels], axis=1
        )

        # Assemble labels that say "all real images".
        misleading_labels = ops.zeros((batch_size, 1))

        # Train the generator (note that we should *not* update the weights
        # of the discriminator)!
        with tf.GradientTape() as tape:
            fake_images = self.generator(random_vector_labels)
            fake_image_and_labels = ops.concatenate(
                [fake_images, image_one_hot_labels], -1
            )
            predictions = self.discriminator(fake_image_and_labels)
            g_loss = self.loss_fn(misleading_labels, predictions)
        grads = tape.gradient(g_loss, self.generator.trainable_weights)
        self.g_optimizer.apply_gradients(zip(grads, self.generator.trainable_weights))

        # Monitor loss.
        self.gen_loss_tracker.update_state(g_loss)
        self.disc_loss_tracker.update_state(d_loss)
```

```
        return {
            "g_loss": self.gen_loss_tracker.result(),
            "d_loss": self.disc_loss_tracker.result(),
        }
```

Train the Conditional GAN:

```
cond_gan = ConditionalGAN(discriminator, generator, latent_dim)
cond_gan.compile(
    d_optimizer=keras.optimizers.Adam(learning_rate=0.0003),
    g_optimizer=keras.optimizers.Adam(learning_rate=0.0003),
    loss_fn=keras.losses.BinaryCrossentropy(from_logits=True),
)

cond_gan.fit(dataset, epochs=20)
```

```
Epoch 1/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 41s 37ms/step - d_loss: 0.5206 - g_loss: 1.2482
Epoch 2/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - d_loss: 0.5620 - g_loss: 1.1221
Epoch 3/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 8s 7ms/step - d_loss: 0.5959 - g_loss: 0.9813
Epoch 4/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - d_loss: 0.6187 - g_loss: 0.9517
Epoch 5/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 10s 9ms/step - d_loss: 0.6353 - g_loss: 0.9269
Epoch 6/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 8s 7ms/step - d_loss: 0.6405 - g_loss: 0.9055
Epoch 7/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - d_loss: 0.6442 - g_loss: 0.8803
Epoch 8/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - d_loss: 0.6476 - g_loss: 0.8794
Epoch 9/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 8s 7ms/step - d_loss: 0.6567 - g_loss: 0.8576
Epoch 10/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 10s 9ms/step - d_loss: 0.6623 - g_loss: 0.8559
Epoch 11/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - d_loss: 0.6582 - g_loss: 0.8483
Epoch 12/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 8s 7ms/step - d_loss: 0.6531 - g_loss: 0.8312
Epoch 13/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - d_loss: 0.6548 - g_loss: 0.8325
Epoch 14/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - d_loss: 0.6655 - g_loss: 0.8463
Epoch 15/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - d_loss: 0.6602 - g_loss: 0.8342
Epoch 16/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - d_loss: 0.6526 - g_loss: 0.8560
Epoch 17/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - d_loss: 0.6594 - g_loss: 0.8560
Epoch 18/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 8s 7ms/step - d_loss: 0.6552 - g_loss: 0.8619
Epoch 19/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 9s 8ms/step - d_loss: 0.6573 - g_loss: 0.8399
Epoch 20/20
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 10s 9ms/step - d_loss: 0.6580 - g_loss: 0.8567
<keras.src.callbacks.history.History at 0x7f3c5a1d4e90>
```

Interpolating between classes with the trained generator:

```
# We first extract the trained generator from our Conditional GAN.
trained_gen = cond_gan.generator

# Choose the number of intermediate images that would be generated in
# between the interpolation + 2 (start and last images).
num_interpolation = 9

# Sample noise for the interpolation.
```

```
interpolation_noise = keras.random.normal(shape=(1, latent_dim))
interpolation_noise = ops.repeat(interpolation_noise, repeats=num_interpolation)
interpolation_noise = ops.reshape(interpolation_noise, (num_interpolation, latent_dim))


def interpolate_class(first_number, second_number):
    # Convert the start and end labels to one-hot encoded vectors.
    first_label = keras.utils.to_categorical([first_number], num_classes)
    second_label = keras.utils.to_categorical([second_number], num_classes)
    first_label = ops.cast(first_label, "float32")
    second_label = ops.cast(second_label, "float32")

    # Calculate the interpolation vector between the two labels.
    percent_second_label = ops.linspace(0, 1, num_interpolation)[:, None]
    percent_second_label = ops.cast(percent_second_label, "float32")
    interpolation_labels = (
        first_label * (1 - percent_second_label) + second_label * percent_second_label
    )

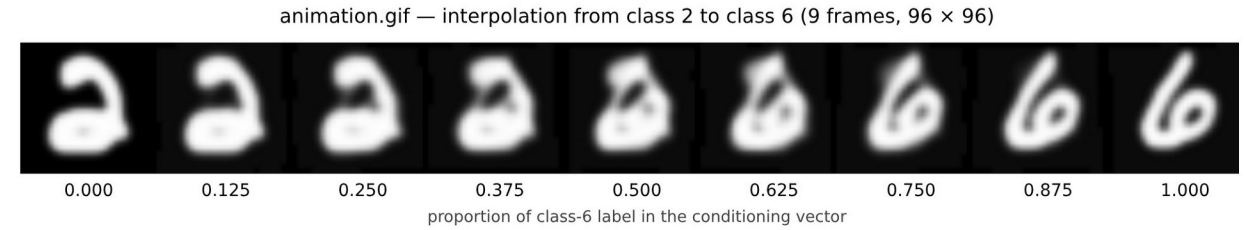
    # Combine the noise and the labels and run inference with the generator.
    noise_and_labels = ops.concatenate([interpolation_noise, interpolation_labels], 1)
    fake = trained_gen.predict(noise_and_labels)
    return fake


start_class = 2
end_class = 6

fake_images = interpolate_class(start_class, end_class)

fake_images *= 255.0
converted_images = fake_images.astype(np.uint8)
converted_images = ops.image.resize(converted_images, (96, 96)).numpy().astype(np.uint8)
imageio.mimsave("animation.gif", converted_images[:, :, :, 0], fps=1)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
```



Generate images for every class label:

```
def generate_for_class(digit, n=10):
    noise = keras.random.normal(shape=(n, latent_dim))
    labels = keras.utils.to_categorical([digit] * n, num_classes)
    return trained_gen.predict(ops.concatenate([noise, labels], 1), verbose=0)

plt.figure(figsize=(8, 8))
plt.suptitle("Generated digits for every class label (row r = digit r)")
for digit in range(10):
    images = generate_for_class(digit)
    for k in range(10):
        plt.subplot(10, 10, digit * 10 + k + 1)
        plt.imshow(images[k, :, :, 0], cmap="gray")
        plt.axis("off")
plt.show()
```



RESULT:

Thus, the Conditional GAN was implemented successfully. By giving the class label to both the Generator and the Discriminator, the trained Generator produced digit images of the requested class and smoothly interpolated between two classes (2 → 6).